

- 概述
 - 概念
 - 起源
 - 分类
 - 按照交换技术分类
 - 电路交换
 - 报文交换
 - 分组交换
 - 1. 电路交换 (Circuit Switching)
 - 2. 报文交换 (Message Switching)
 - 3. 分组交换 (Packet Switching)
 - 三者对比小结
 - 按照网络范围分类
 - 按照拓扑结构分类
 - 协议|协议栈
 - 协议
 - 协议栈：协议的层次结构
 - 吞吐量
 - 四大时延
- 应用层（数据、报文）
 - 1. 架构
 - 端系统
 - C/S
 - 客户端
 - 服务器
 - P2P
 - 混合结构
 - 2. 协议
 - 应用
 - 标准网络应用：
 - 专用应用层协议：
 - 选择传输层协议
 - TCP协议提供的服务： 功能完善、 面向连接、 可靠传输服务
 - UDP协议提供的服务： 简单、 高效传输服务
 - 网络应用选择的传输层协议
 - 3. HTTP
 - C/S模式

- 响应格式
 - HTTP报文结构
 - 请求报文
 - 响应报文
- HEAD/Body模式
- 请求
 - GET
 - POST
- 状态码
- 常见状态码
- 无状态连接
- HTTP状态-持续连接和非持续连接
 - 非持续连接（HTTP/1.0）
 - 持续连接(HTTP/1.1)
- URL
- 4.FTP
 - 状态
 - 端口
 - C/S模式
 - TFTP
- 5.Email
 -  一张图先概览（邮件发送/接收流程）
 - 用户代理（UA）
 - SMTP（发邮件）
 - 用户 --> 服务器
 - 服务器 --> 服务器
 -  1. SMTP（Simple Mail Transfer Protocol）
 -  2. HTTP（只用于 Web 邮箱界面）
 - 服务器 --> 用户|POP3（收邮件）
-  二、接收邮件时：POP vs IMAP?
 -  1. POP3（Post Office Protocol v3）
 -  2. IMAP（Internet Message Access Protocol）
-  总结一张表：邮件协议使用对比
- 特别注意，HTTP既可以收也可以发邮件
-  记忆口诀：
- 6.DNS
 - 查询
 - 本地代理（DNS缓存）

- 例题
 - 其中的非持续连接、流水线方式，都会导致不同的RTT，务必注意！
HTML文件套文件是要访问之后才知道的！
-  题干信息总结：
-  通用前置时间：DNS 查询时间
-  1. 非持续连接（Non-persistent HTTP）
-  2. 持续连接（Persistent HTTP），非流水线
-  3. 持续连接 + 流水线（Pipelining）
-  最终三种模式对比表：
- 7.P2P
 - 1. 对等（Peer-to-Peer）
 - 2. 可扩展（Scalability）
 - 3. 瓶颈（Bottleneck）
 - 4. 版权（Copyright/Legal Issues）
 - 小结
- 传输层(报文段[TCP、Segment]、数据报[UDP、Datagram])
 - 复用
 - 端口号(16bit)
 - UDP
 - 伪首部
 - UDP数据报格式
 - UDP校验和计算
 - 可靠数据传输原理
 - 停止等待协议
 - 流水线协议
 - 重传协议
 - GBN（Go-Back-N，回退N帧法）
 - 特别注意为了防止串数据，窗口的大小是 2^k-1
 - SR（Selective Repeat，选择重传法）
 - 特别注意为了防止串数据，窗口的大小是 $2^{(k-1)}$
 - TCP（报文段）
 - TCP报文段格式
 - 三次握手、四次挥手
 - 三次握手
 - 四次挥手
 - TCP流量控制
 - 定时器
 - 滑动窗口

- 零窗口
 - RTT估计
 - TCP拥塞控制
 - 发送窗口 = Min (通知窗口(rwnd), 拥塞窗口(cwnd))
 - 公平性
- 网络层 (分组(Packet))
 - 功能
 - IP地址
 - 分类
 - 特殊IP地址
 - 专用IP地址
 - 子网划分
 - 子网掩码 (Subnet Mask)
 - CIDR
 - NAT
 - 1. IP 层的修改
 - 2. 传输层的修改
 - 3. 应用层的潜在影响
 - 4. NAT 带来的副作用
 - 简单流程示意
 - IP格式
 - 第一行
 - 版本字段:
 - 分组头长度: 占4bit,
 - 总长度:
 - 第二行
 - 标识:
 - 标志:
 - 段偏移:
 - 第三行
 - 生存时间TTL:
 - 协议:
 - 头部校验和:
 - MTU
 - 分组分段
 - 例子1
 - 例子2
 - 例子3

- IP路由选择
 - 静态路由、动态路由
 - 静态路由选择算法：
 - 动态路由选择算法：
 - 路由聚合|最长前缀匹配
- 路由协议
 - RIP(Bellman-Ford算法) (UDP)
 - 特别注意路由向量中距离大于等于16的表明路由不可达！！
 - OSPF (Dijkstra算法)
 - 内部算法对比
 - BGP (外部网关)
- ICMP (IP直装)
 - 检查
 - 查询
 - 源点抑制
- 3. 工作流程
 - 种类
- IPV6
 - 结构
 - 压缩
 - CIDR
 - 特殊地址
 - 简要使用场景
 - IPv6与IPv4的相同点
- DHCP
- 数据链路层 (帧)
 - 帧
 - 确定帧的界限
 - 重传(同网络层的GBN和SR,也接受ACK)
 - 一、链路层的复用 (多址／多路访问)
 - 复用量
 - 1. 时分复用 (TDM)
 - 2. 频分复用 (FDM)
 - 3. 码分复用 (CDM/CDMA)
 - 为什么公式长这样？
 - 单工、半双工与全双工
 - 二、链路层的“邻接传输”
 - 三、点对点协议：PPP

- 四、链路层差错控制：ARQ
- 以太网(以下涉及到的概念都是以太网相关的)
- MAC
 - 局部网络地址,不越过路由器|网关
- 五、典型共享介质多址协议
- ARP|RARP
 - ARP(IP-->MAC)
 - RARP(MAC-->IP)
- 纠错
 - CRC
- 必考
 - 奇偶校验
 - 海明码
 - 1. 误码率 (Bit Error Rate, BER)
 - 2. 信噪比 (Signal-to-Noise Ratio, SNR)
- 二、二进制指数退避算法 (Binary Exponential Backoff)
 - 算法流程图示意
- 物理层(比特流)
 - 调制|解调
 - 调制 (Modulation)
 - 解调 (Demodulation)
 - 调制方法
 - 调幅 (Amplitude Modulation, AM)
 - 调频 (Frequency Modulation, FM)
 - 调相 (Phase Modulation, PM)
 - 脉冲编码调制 (Pulse Code Modulation, PCM) ,重要!!!
 - 步骤是采样、量化、编码!!
 - 数字信号编码
 - 1. 非归零编码 (Non-Return-to-Zero, NRZ)
 - 2. 归零编码 (Return-to-Zero, RZ)
 - 3. 曼彻斯特编码 (Manchester Encoding)
 - 4. 差分曼彻斯特编码 (Differential Manchester Encoding)
 - 传输介质
 - 1. 双绞线 (Twisted Pair Cable)
 - 2. 同轴电缆 (Coaxial Cable)
 - 3. 光纤 (Fiber Optic Cable)
 - 4. 无线传输介质
 - 香农定理

- 一、信道容量公式
- 四、简要示例
- 复用
 - 1. 频分复用 (Frequency Division Multiplexing, FDM)
 - 2. 时分复用 (Time Division Multiplexing, TDM)
 - 同步时分多路复用
 - 统计时分多路复用
 - 3. 波分复用 (Wavelength Division Multiplexing, WDM)

概述

概念

计算机网络：由若干结点（计算机、路由器等）和连接这些结点的链路组成的按功能划分的集合
也就是说,单独运行,分开,链路连接,传播数据

起源

Internet起源于ARPANET

分类

按照交换技术分类

电路交换

报文交换

分组交换

下面对三种经典的交换方式作一并列介绍，并给出它们的主要特点、优缺点及典型应用场景。

1. 电路交换（Circuit Switching）

原理

- 在通信开始前，发送方与接收方之间在网络中**建立一条专用的、端到端的物理或逻辑链路**；

2. 报文交换（Message Switching）

原理

- 将整个用户报文当作一个整体，在**每个交换节点上完整存储—转发**（store-and-forward）；

3. 分组交换（Packet Switching）

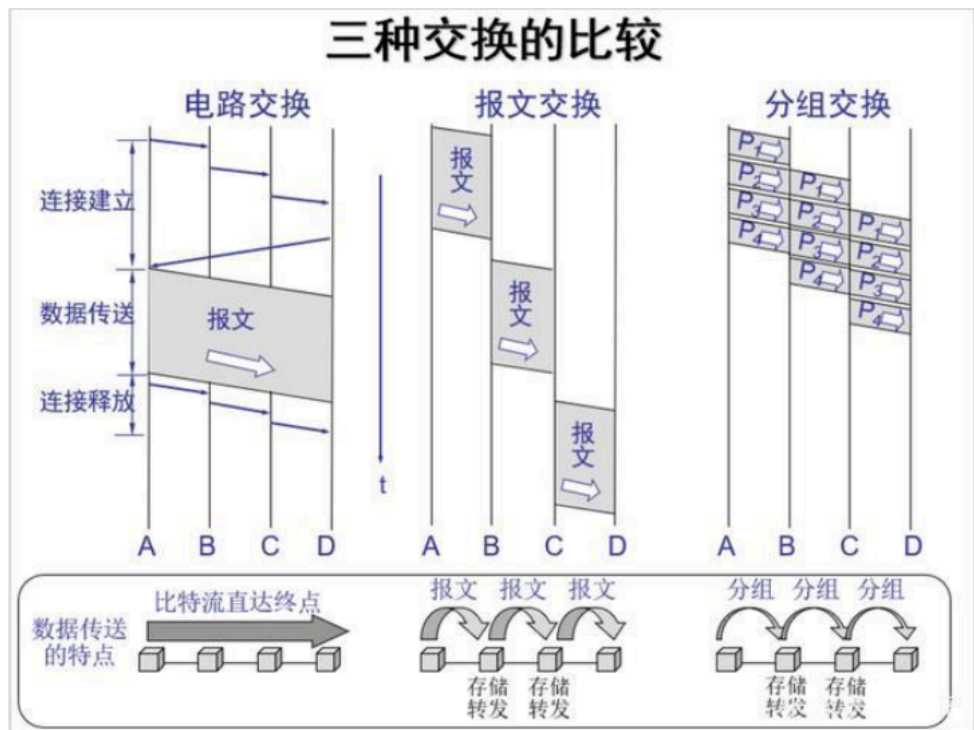
原理

将报文切分为若干固定长度或可变长度的**分组（packet）**，每个分组独立携带目的地信息

三者对比小结

特性	电路交换	报文交换	分组交换
建路方式	预先建立专用电路	存储—转发整个报文	存储—转发分组
时延类型	固定、低	随报文长度增加	随网络拥塞动态变化
带宽利用率	低	中	高
节点缓存需求	无	高	适中
适用流量特征	持续、实时业务	大量完整消息一次性	突发、多业务混合

五、网络交换技术

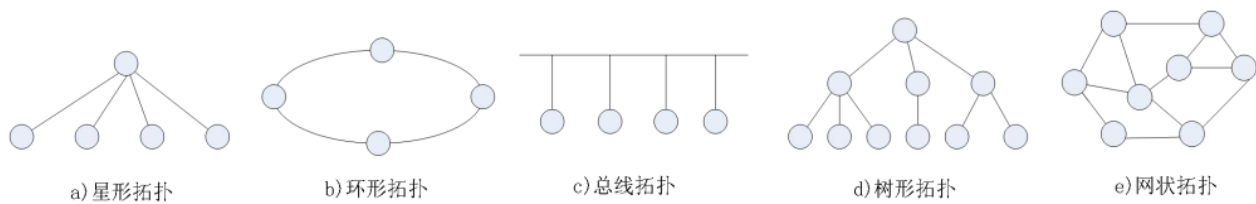


按照网络范围分类

- 广域网(WAN)
- 局域网(LAN)
- 城域网(CAN)
- 个人区域网(PAN)

按照拓扑结构分类

- 总线型
- 环型
- 星型
- 网状型



协议|协议栈

协议

- 语义
- 语法
- 时序

协议栈：协议的层次结构

从上到下：应用层、传输层、网络层、数据链路层、物理层

5. 原理性的网络体系结构

- (1) 物理层
 - 规定物理接口，比特流透明传输。
- (2) 数据链路层
 - 实现逻辑链路上无差错的数据帧传输。
- (3) 网络层
 - 实现网络分组传输，解决寻址、路由、转发。
- (4) 传输层
 - 实现进程之间可靠/无差错的端到端通信。
- (5) 应用层
 - 根据应用进程通信要求，满足用户的需要。



原理性的网络体系结构

吞吐量

- **瞬时吞吐量**

$$T = \frac{\text{传输到达的数据量 (bits 或 bytes)}}{\text{所用时间 (seconds)}} \quad \left[\text{bps, Bps} \right]$$

- **平均吞吐量**

在一个较长时段内测量多次瞬时吞吐量的均值。

四大时延

1. **传播时延** (Transmission Delay)

- 数据在链路上传播所需的时间，取决于链路长度和信号传播速度。
- 计算公式：
$$\text{传播时延} = \frac{\text{链路长度}}{\text{信号传播速度}}$$

2. **排队时延** (Queuing Delay)

- 数据包在路由器或交换机的队列中等待转发的时间，受网络拥塞影响。

3. **处理时延** (Processing Delay)

- 数据包在路由器或交换机中被处理（如查找路由表、进行错误检查等）所需的时间。

4. **传输时延** (Transmission Delay)

- 数据包在链路上传输所需的时间，取决于数据包大小和链路带宽。

- **时延** (Delay)

- 时延 (delay)是指数据（一个报文或分组）从网络（或链路）的一端传送到另一端所需的时间，也称为延迟或迟延

- **四大时延**

- **传输时延**(transmission delay)：数据从结点进入到传输媒体所需要的时间，传输时延又称为发送时延（信道能力）
- **传播时延**(propagation delay)：电磁波在信道中需要传播一定距离而花费的时间（介质本身）
- **处理时延**(processing delay)：主机或路由器在收到分组时，为处理分组（例如分析首部、提取数据、差错检验或查找路由）所花费的时间
- **排队时延**(queueing delay)：分组在路由器输入输出队列中排队等待处理所经历的时延

应用层（数据、报文）

1.架构

端系统

端系统是指网络中网络边缘部分的用户设备。概念上等同于主机，也就是运行各种程序的东西

C/S

比如说email和Web。简单来说，就是服务器一直运行等着客户端

客户端

- 与服务器通信，使用服务器提供的服务
- 间歇性接入网络
- 可能使用动态IP
- 不会与其它客户机直接通信

服务器

- 不间断提供服务
- 永久性访问地址/域名
- 利用大量服务器实现可扩展性

P2P

P2P对等方式是指两个进程在通信时并不区分服务的请求方和服务的提供方

- 只要两个主机都运行P2P软件，它们就可以进行平等、对等的通信
- 双方都可以下载对方的内容
- 没有永远在线的服务器
- 任意端系统/节点可以通信
- 节点间歇性接入网络

- 节点可能改变IP

混合结构

apster

- 文件传输采用P2P
- 文件的搜索采用C/S

2.协议

进程间通信

- 用**套接字**进行通信
- 复用对象为**应用**
- 内容为**数据、报文**

应用

E-mail、FTP、TELNET、Web、IM、IPTV、VoIP

标准网络应用：

E-mail、FTP、TELNET、Web等

专用应用层协议：

很多P2P共享文件的应用层协议属于专用协议（P2P文件分发协议：BitTorrent）

选择传输层协议

TCP协议提供的服务：功能完善、面向连接、可靠传输服务

- 支持可靠的面向连接服务
- 支持字节流传输服务
- 支持全双工服务
- 进程在什么时间、如何发送报文，以及如何响应

UDP协议提供的服务：简单、高效传输服务

- 无连接、不可靠的传输
- 无提供拥塞控制机制
- 不提供最小延时保证

网络应用选择的传输层协议

网络应用类型	应用层协议	传输层协议
E-mail	SMTP	TCP
TELNET	TELNET	TCP
Web	HTTP	TCP
FTP	FTP	TCP
DNS	DNS	UDP或TCP
流媒体	Real Network	UDP或TCP
VoIP	Net2phone	UDP

3.HTTP

C/S模式

响应格式

纯文本存储

HTTP报文结构

■ HTTP请求报文结构

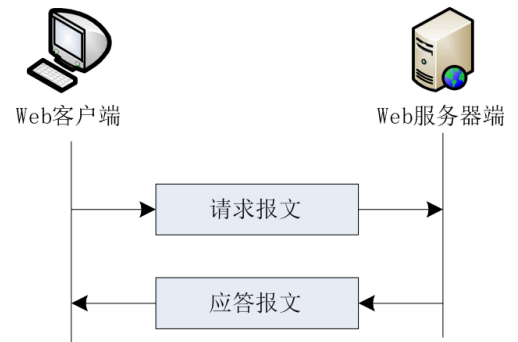
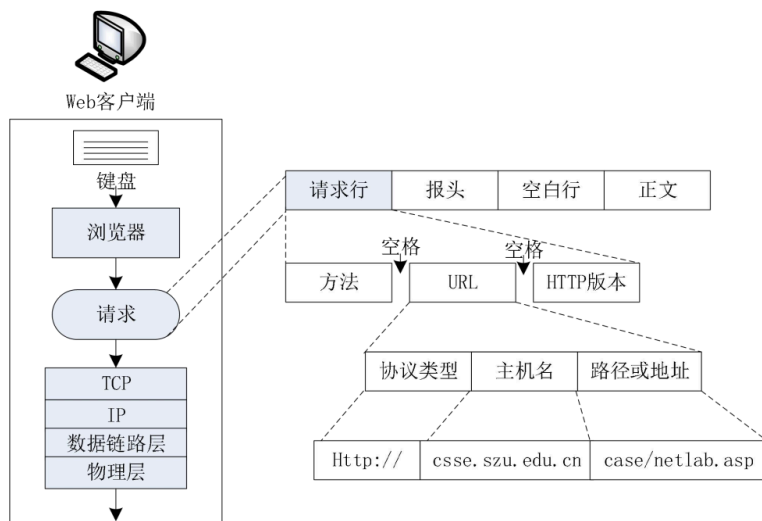


图4-27 HTTP协议请求与应答的过程

空白行就是下面的CRLF <Start - line>\r\n

: \r\n

: \r\n ... \r\n // 空行 (仅 CRLF)，标志首部结束 CRLF: 代表回车换行符序列 \r\n (ASCII 13 + 10)，这是 HTTP/1.x 中唯一认可的“换行”方式。

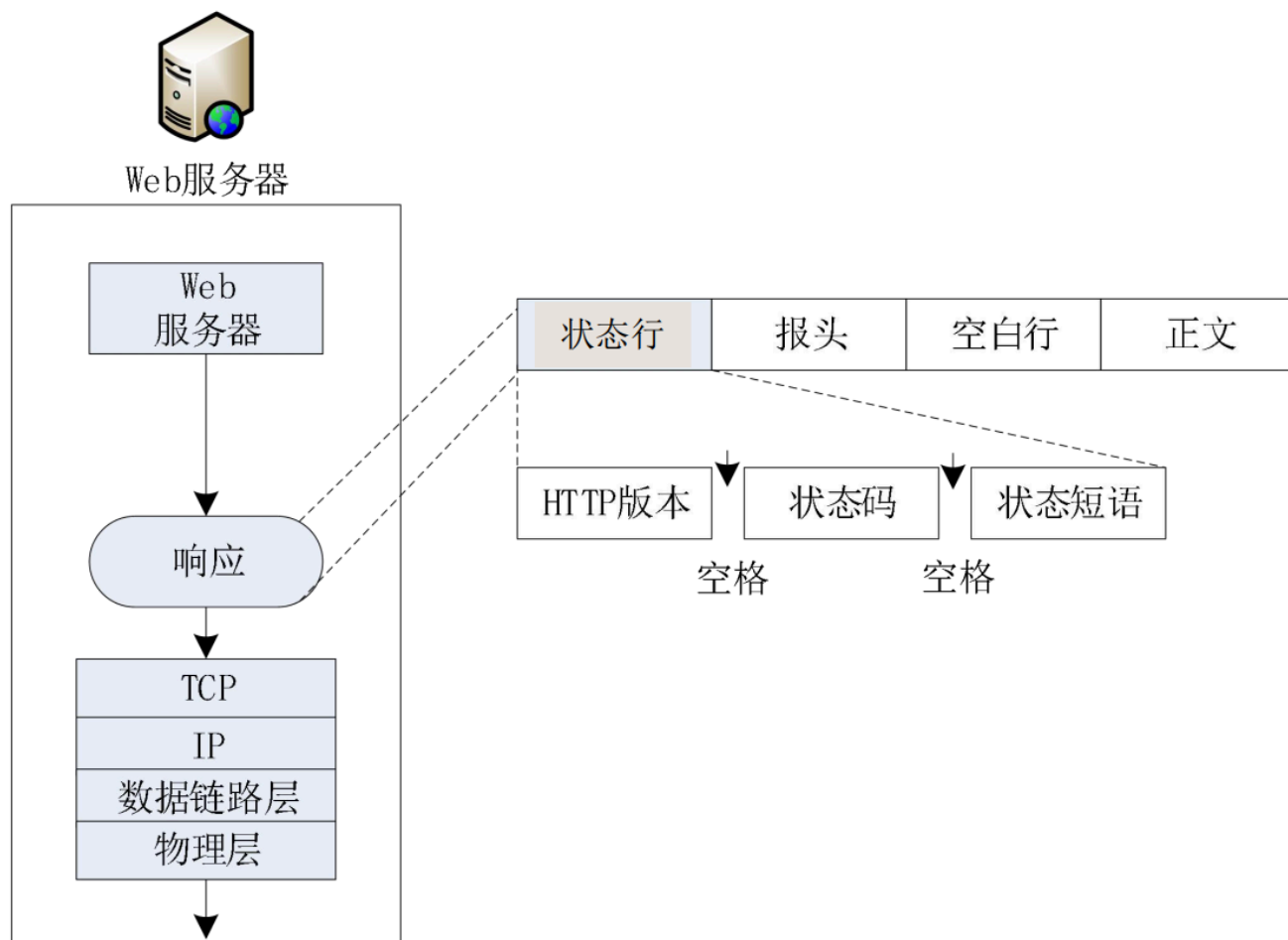
空行 (单独的 \r\n) 用来分隔 首部部分 和 消息体部分 **也就是说，每个消息行必须要用 CRLF结束**

请求报文

```
GET /index.html HTTP/1.1\r\nHost: www.example.com\r\nConnection: close\r\nUser-Agent: Mozilla/5.0\r\n\r\n
```

响应报文

构



HTTP/1.1 200 OK\r\n Content-Type: text/html\r\n Content-Length: 1024\r\n \r\n ...

HEAD/Body模式

即在上述的请求头(HEAD)后附上具体的传输内容(body) 用\r\n分隔

请求

▪ 请求报文、应答报文的报头结构



报头结构

GET

```
GET /index.html HTTP/1.1\r\n
Host: www.example.com\r\n
Connection: close\r\n
User-Agent: Mozilla/5.0\r\n
\r\n
```

POST

```
POST /index.html HTTP/1.1\r\n
Host: www.exam
```

状态码

- 1xx：信息，服务器收到请求，需要请求者继续执行操作
- 2xx：成功，操作被成功接收并处理
- 3xx：重定向，需要进一步的操作以完成请求
- 4xx：客户端错误，请求包含语法错误或无法完成请求

- 5xx：服务器错误，服务器在处理请求的过程中发生了错误

▪ HTTP应答报文结构

代码	短语	说明
100	Continue	请求的开始部分已经被接受，客户可以继续他的请求
101	Switching	服务器同意客户的请求，切换到更新报头中定义的协议
200	Ok	请求成功
201	Created	新的URL被创建
202	Accepted	请求被接受，但还没有马上起作用
204	No accepted	报文中没有内容
301	Multiple choices	所请求的URL指向多个资源
302	Moved permanently	服务器已经不再使用所使用的URL
304	Moved temporarily	所请求的URL已暂时地移走
400	Bad request	在请求中有语法错误
401	Unauthorized	请求缺乏适当的授权
403	Forbidden	服务被拒绝
404	Not found	文档未发现
405	Method not allowed	URL不支持
406	Not acceptable	所请求的格式不可接受
500	Server error	服务器端出错
501	Not implemented	所请求的动作不能完成
503	Service unavailable	服务器暂时不可使用，但以后可能接受请求

➤ 状态码都是三位数字

- 1xx 表示通知信息的，如请求收到了或正在进行处理。
- 2xx 表示成功，如接受或知道了。
- 3xx 表示重定向，表示要完成请求还必须采取进一步的行动。
- 4xx 表示客户的差错，如请求中有错误的语法或不能完成。
- 5xx 表示服务器的差错，如服务器失效无法完成请求。

常见状态码

- 200 OK：请求成功。一般用于GET和POST请求
- 301 Moved Permanently：请求的资源已被永久移动到新位置

无状态连接

HTTP为**无状态协议**，服务器用**cookies**保持用户状态

- Cookie文件保存在用户的主机中，内容是服务器返回的一些附加信息，由用户主机中的浏览器管理
- Web服务器建立后端数据库，记录用户信息，cookie作为关键字 简单说，就是用cookie认用户和它的状态

HTTP状态-持续连接和非持续连接

非持续连接（HTTP/1.0）

- 每个请求/响应对应一个TCP连接
- 服务器处理完请求后关闭TCP连接
- 适用于少量数据传输

持续连接(HTTP/1.1)

- 服务器在处理完一个请求后保持该TCP连接打开
- 服务器在等待后续的请求

URL

▪ URL与信息资源定位

- 访问Web网站要使用的HTTP协议，其形式为：
- http://服务器名[:端口号]/路径/文件名
- URL（:冒号左边）指明了URL的访问方式
- HTTP的默认端口号是80（可以省略）
- 路径/文件名用于直接指向服务器中的某一个文件
- 省略路径和文件名，则URL就指向了Internet上的某个主页

4.FTP

用于传文件

由于明文传输用户账号密令，不安全被弃用

状态

有状态连接

- 可以指定数据类型
- 可以指定传输模式
- 可以指定文件结构
- 可以鉴别用户身份

端口

- 20：数据传输端口
- 21：控制端口 控制端口先于数据端口被建立，后于数据端口被释放

C/S模式

- FTP控制命令

 - USER：向服务器发送用户名
 - PASS：向服务器发送用户口令
 - LIST：向服务器请求发送当前目录的文件列表
 - RETR（filename）：从服务器端检索指定的文件并取得它的副本
 - STOR（filename）：将客户主机的一个文件存储到FTP服务器中
- FTP响应命令

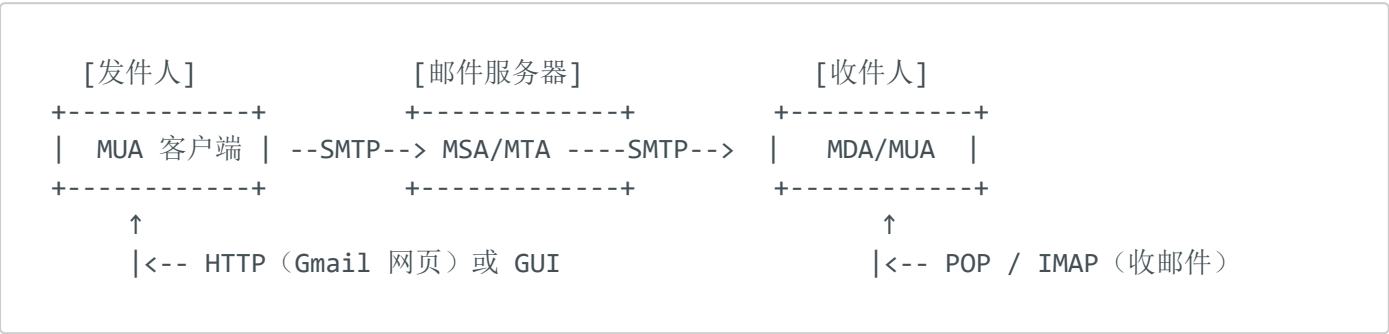
 - 125：数据连接正确，准备传输文件
 - 150：数据连接即将打开
 - 220：服务就绪
 - 221：服务关闭
 - 226：数据连接关闭
 - 230：用户注册完成
 - 331：用户名正确，需要传输用户口令

TFTP

使用UDP协议，69端口,无状态连接，不鉴别用户，不能上传

5.Email

✅ 一张图先概览（邮件发送/接收流程）



用户代理（UA）

在电子邮件系统里，“用户代理”（User Agent）就是你用来撰写、发送、接收和阅读邮件的软件，也常简称为 MUA（Mail User Agent）。

- MUA（Mail User Agent）：邮件用户代理

- 运行在你电脑或手机上的邮件客户端。
- 典型例子：Outlook、Thunderbird、Apple Mail、Foxmail、Gmail 网页版、手机自带的 Mail 应用等。
- 它负责和邮件服务器（MSA/MTA、MDA）打交道，把你写的邮件发出去，也把收到的邮件拉下来或展示给你看。

当你查看邮件原始头部（raw headers）时，如果看到像这样的一行：

```
User-Agent: Mozilla Thunderbird 115.3.1
```

那就表示 **发件人当时使用的就是 Thunderbird 115.3.1 这个客户端**。

一句话记住：

Email 的“用户代理” = 你用来查看／写／发邮件的客户端程序（MUA）。

SMTP（发邮件）

简单邮件传输协议使用TCP连接，25端口 使用ASCLL文本传输，如果要传其他东西比如说图片要用Base64或者UTF-7转化

■ 扩展：Base64和UTF-7

- Base64
 - 将二进制数据转换为ASCII字符的编码
 - 它将每3个字节的数据转换为4个可打印字符的ASCII字符串
 - 可以在只能传输文本的协议中传输数据
- UTF-7
 - 用于在7位传输协议中表示Unicode字符（邮件）
 - 基本原则是将Unicode字符转换为Base64编码形式
 - 例如：将字符串"Hello, 你好！"转换为UTF-16编码：
 - "Hello, 你好！" 的UTF-16编码为：00480065006C006F002C0020004F6000E01FF01
 - 将UTF-16编码的数据转换为Base64编码：
 - Base64编码后的数据为：SGVsbG8sIOeah+e6vQ
 - 使用UTF-7的转义机制来表示Base64编码的数据：
 - 转义后的数据为：+SGVsbG8sIOeah+e6vQ--

用户 --> 服务器

- 手机软件等都用的是**SMTP**
- Web网页用的是**HTTP**

服务器 --> 服务器

使用SMTP



1.

SMTP (Simple Mail Transfer Protocol)

用于从发件人客户端 → 邮件服务器 → 收件人服务器 的邮件传输过程

使用 SMTP 的情况

使用邮件客户端软件 (Outlook、Thunderbird、Foxmail等)

服务器之间转发邮件

程序自动发邮件 (用 SMTP 发验证邮件)

✦ **关键点：**SMTP 是“发邮件”的**底层传输协议**，通常是客户端配置时写的 `smtp.example.com`。



2. HTTP (只用于 Web 邮箱界面)

你打开 Gmail / QQ 邮箱 / Outlook.com 浏览器网页，发邮件时，其实是前端调用 Web API (HTTP POST 请求) 来提交邮件内容

使用 HTTP 的情况

你在网页中写邮件、发邮件 (比如 Gmail)

移动 App 或 Web Mail 使用 API 发送邮件

✦ **关键点：**HTTP 只用于“用户界面”的交互，底层服务器还是转成 SMTP 发出去的。

服务器 --> 用户|POP3 (收邮件)



二、接收邮件时：POP vs IMAP?

邮件已经存储在收件人服务器上，客户端要“取邮件”，这时用的就是 POP 或 IMAP。

✓ 1. POP3（Post Office Protocol v3）

特点

下载邮件后默认**删除服务器副本**（可设置保留）

不同步“已读”、“已删”等状态

更适合单设备使用（老手机/老电脑）

✦ 使用场景：

- 你只用一个客户端（比如 Outlook）看邮件
- 不需要多设备同步
- 希望节省服务器空间

✓ 2. IMAP（Internet Message Access Protocol）

特点

所有邮件保留在服务器

支持同步“已读、未读、分类”等状态

适合多设备同步（如电脑+手机+平板）

✦ 使用场景：

- 你要在多个设备上同步邮件状态
- 用 Web 邮箱 + App 邮箱交替查看
- 云端统一管理邮件（现代主流推荐）

● 总结一张表：邮件协议使用对比

动作	协议	用途	使用场景
发邮件	SMTP	发送邮件	客户端发邮件 / 服务器转发
发邮件	HTTP	网页邮箱界面发邮件	浏览器打开 Gmail 发邮件
收邮件	POP3	下载邮件	单设备、脱机查看
收邮件	IMAP	同步邮件	多设备、多客户端同步查看
收邮件	HTTP	网页邮箱界面收邮件	浏览器打开 Gmail 收邮件

特别注意，HTTP既可以收也可以发邮件



记忆口诀：

发邮件靠 SMTP，网页靠 HTTP，收邮件用 POP/IMAP，看你设备几台机。

6.DNS

域名—>IP地址 用的UDP协议，53端口

查询

- 递归查询：DNS服务器直接返回查询结果,具体细节的服务器查询交给**第一个访问的服务器**
- 迭代查询：DNS服务器返回下一个查询的DNS服务器地址，**自行查询**，常用

本地代理（DNS缓存）

即**DNS缓存**，本地存储有有就不查了，**直接读取**

例题

- 假设在浏览器上点击一个HTML网页的URL，但这个网页的IP地址没有缓存在本地主机上。假设要解析取到这个网页的IP地址，需要以反复查询的方式查询 n 个DNS服务器（往返时间分别为 $RTT_1, RTT_2, RTT_3, \dots, RTT_n$ ）。这个HTML网页中链接了3张图片。假设网页和图片都很小（即忽略他们的传输时间），且从主机发出请求到取回某个小文件的往返时间是 RTT_w 。在下面三种情况下，客户端读取这些对象所需的时间是多少？

1. 非持续HTTP连接；
2. 持续HTTP连接，非流水线方式；
3. 持续HTTP连接，流水线方式。

其中的非持续连接、流水线方式，都会导致不同的RTT，务必注意！HTML文件套文件是要访问之后才知道的！

这是一道非常经典的网络延迟分析题，我们来逐步解析：

题干信息总结：

- 要解析 IP 地址，需要经过 n 个 DNS 查询，每个查询往返时间为 $RTT_1 \sim RTT_n$ ；
- HTML 页面中有 1 个 HTML 文件 + 3 张小图片；
- 每个 HTTP 请求获取一个小对象，往返时间为 RTT_w ；
- 忽略数据传输时间，即只考虑控制层面的时间消耗；
- 分析三种连接模式下客户端从点击 URL → 全部文件加载完毕所需时间。

通用前置时间：DNS 查询时间

无论哪种连接方式，最开始都要先完成 DNS 解析：

$$\text{DNS 总时延} = RTT_1 + RTT_2 + \dots + RTT_n$$

我们用一个总和记为：

$$T_{\text{DNS}} = \sum RTT_i \quad (i=1 \text{ 到 } n)$$

✓ 1. 非持续连接（Non-persistent HTTP）

特点：

- 每个对象都要单独建立 TCP 连接（3 次握手），发送请求、接收响应，然后关闭连接；
- 一个对象 = 一个完整的 HTTP 往返 + TCP 建连。

每个对象的总延迟 = $2 \times RTT_w$ （一次握手+一次请求/响应）

有 1 个 HTML + 3 张图片，共 4 个对象

所以总时延是：

$$\begin{aligned} T &= T_{\text{DNS}} + 4 \times 2RTT_w \\ &= T_{\text{DNS}} + 8RTT_w \end{aligned}$$

✓ 2. 持续连接（Persistent HTTP），非流水线

特点：

- 一个 TCP 连接传多个对象，但是“**串行处理**”：一个请求 - 等响应完 - 再发下一个；
- 只建立一次 TCP 连接（3 次握手），后续复用。

时延：

- 建立连接： $1 \times RTT_w$

- 第一个请求到响应： $1 \times RTT_w$
- 后续每个对象都要等上一个结束后再发，且每个需要 $1 \times RTT_w$

总对象数 4 (HTML+3图)，连接共用，不用重复建 TCP 连接。

$$\begin{aligned} T &= T_{\text{DNS}} + 1RTT_w \text{ (建连接)} + 4RTT_w \text{ (串行请求4个对象)} \\ &= T_{\text{DNS}} + 5RTT_w \end{aligned}$$

✓ 3. 持续连接 + 流水线 (Pipelining)

特点：

- 一个 TCP 连接
- **请求并发发出** (不等上一个返回就能发下一个)；
- 响应按顺序返回，但客户端几乎可以“一口气发完所有请求”。

时延：

- 建立 TCP 连接： $1 \times RTT_w$
- 发出所有请求几乎不占时间，主要等第一个响应回来 ($1 \times RTT_w$)，后续响应连续回来，不增加额外时延

因此：

$$\begin{aligned} T &= T_{\text{DNS}} + 1RTT_w \text{ (建连接)} + 1RTT_w \text{ (收完所有响应)} \\ &= T_{\text{DNS}} + 2RTT_w \end{aligned}$$

最终三种模式对比表：

模式	解释	总时延
非持续连接	每个对象建连接	$T_{\text{DNS}} + 8RTT_w$
持续连接 (非流水线)	一个连接串行发请求	$T_{\text{DNS}} + 5RTT_w$

模式	解释	总时延
持续连接 + 流水线	一个连接并发请求	$T_{DNS} + 2RTT_w$

7.P2P

下面我从“对等”、“可扩展”、“瓶颈”、“版权”四个方面，结合 P2P（点对点）网络特点做一个详细讲解。

1. 对等（Peer-to-Peer）

- **定义** 在 P2P 架构中，网络中的节点（Peer）既是客户端也可以是服务器——它们相互平等，没有固定的中心服务器来管理或分发资源。
 - **特点**
 - 节点之间直接通信，请求资源／提供资源都由节点自己完成。
 - 没有单点故障：任何一个节点挂掉都不会让整个系统瘫痪。
 - **举例**
 - BitTorrent：下载方同时也向其他下载者上传已经拥有的文件块。
 - Skype（早期）：通话时音视频流在用户之间直接转发。
 - **优劣**
 - 优点：弹性高、抗攻击；网络越大，资源越丰富。
 - 缺点：管理和协调比较复杂，易出现“搭便车”（free-riding）现象。
-

2. 可扩展（Scalability）

- **定义** P2P 网络随着节点数增加，理论上整体容量和带宽也线性增长，能够更好地支撑大规模用户。
- **实现机制**

- **分布式哈希表 (DHT)**: 如 Kademlia, 将资源标识映射到网络中的不同节点, 实现高效定位。
- **分片下载**: 大文件被切成多个块, 不同节点并行下载, 加速总体传输。

- **优点**

- 资源和带宽集中度低: 用户越多, 上传能力越强, 总体下载速度通常也越高。
- 没有中心服务器压力瓶颈。

- **注意点**

- 网络拓扑的维护 (如 DHT 中的路由表更新) 需要额外通信开销。
- 节点离线频繁时, 分布式索引和资源可用性会受到影响。

3. 瓶颈 (Bottleneck)

- **可能的瓶颈点**

1. **节点上线／离线不稳定**

- 节点随时可能下线, 导致某些资源块瞬间不可用。

2. **网络延迟与带宽差异**

- 某些节点带宽很小, 成了慢速上传源; 高延迟节点影响请求路由效率。

3. **协调和发现开销**

- DHT 查找路径需要多跳; 节点越多, 维护路由表和探测活跃节点的成本越高。

- **缓解策略**

- **引入超节点 (Supernode)**: 选取部分高带宽／高稳定性节点承担更多索引任务 (如早期的 Kazaa、Skype)。
- **多点复制**: 同一个资源在多个节点保存, 避免单点不可用。
- **动态优化路由**: 根据延迟、带宽等指标调整查找路径, 优先选择高速节点。

4. 版权 (Copyright／Legal Issues)

- **背景** P2P 技术本身中立，但由于便于大规模文件（尤其是影视、音乐、软件）分发，常被用于侵权分享。
 - **风险点**
 - 用户在未经授权的情况下分享受版权保护的作品。
 - 没有中心化监控，版权方难以及时追踪并阻止侵权。
 - **应对措施**
 - **法律手段**：版权方通过 DMCA 等法规，对侵权节点发出下架或屏蔽通知。
 - **协议增强**：有些 P2P 平台内置“合法内容库”分类，并对侵权内容进行自动过滤／下架。
 - **内容指纹与追踪**：通过文件指纹、块哈希识别已知侵权资源，并加入黑名单。
 - **正面应用**
 - 开源软件（Linux 发行版镜像）通过 P2P 分发，节省服务器带宽。
 - 公有领域资料／教育资源在 P2P 上合法共享。
-

小结

- ****P2P 的“对等”****造就了无中心、抗故障的网络结构；
- **可扩展性**依赖分布式索引与并行下载机制；
- **瓶颈**多在网络不稳定与索引维护；
- **版权问题**是 P2P 演进过程中必须面对的重要法律与合规挑战。

传输层(报文段[TCP、Segment]、数据报[UDP、Datagram])

实现主机进程间可靠服务，实现“端到端”服务（主机-主机 → 端-端）
也就是连接进程
提供尽力而为的服务

复用

发送端多路复用: 传输层从多个套接字收集数据，交给网络层发送，多个TCP使用同一个IP进行传输。

接收端多路分解: 传输层将从网络层收到的数据，交付给正确的套接字 **端口号**标识进程
进程通信标识五元组: 协议、本地地址、本地端口、远程地址、远程端口号，也叫全相关

端口号(16bit)

- **熟知端口号:** 给每种服务器分配的确定的全局端口号，也叫公认端口号，范围在0~1023，统一分配与控制
- **临时端口号:** 客户端程序使用的临时端口号，由客户端上TCP/IP软件随机选取，范围在49152~65535
- **注册端口号:** 在IANA（国际因特网地址分配委员会）注册的端口号，没有明确的定义服务对象，不同程序可根据实际需要自己定义，范围在1024~49152 **443 HTTPS**

表5-1 UDP常用的熟知端口号			表5-2 TCP常用的熟知端口号		
端口号	服务进程	说 明	端口号	服务进程	说 明
53	Domain	域名服务	20	FTP	文件传输（数据连接）
67/68	DHCP	动态主机配置协议	21	FTP	文件传输（控制连接）
69	TFTP	简单文件传输协议	23	TELNET	网络虚拟终端协议
111	RPC	远程过程调用	25	SMTP	简单邮件传输协议
123	NTP	网络时间协议	80	HTTP	超文本传输协议
161/162	SNMP	简单网络管理协议	119	NNTP	网络新闻传输协议
520	RIP	路由信息协议	179	BGP	边界路由协议

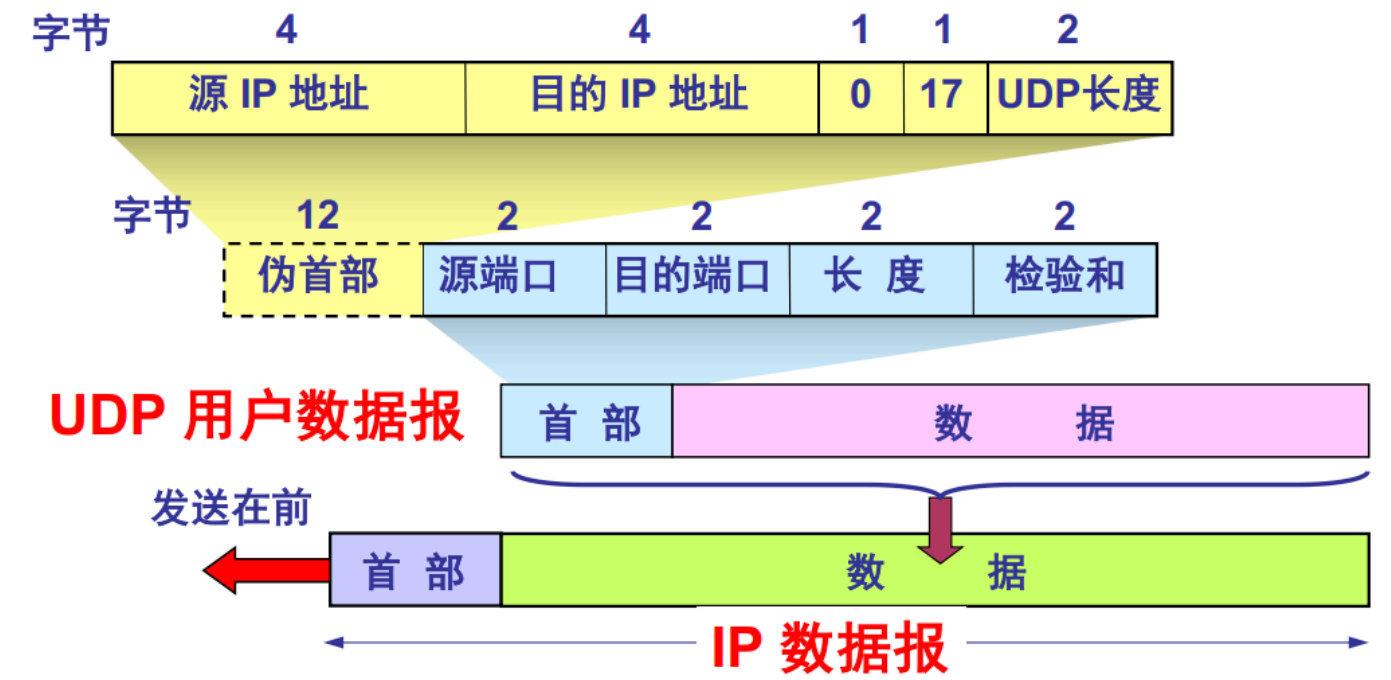
UDP

伪首部

无连接，简单、快速 在计算**校验和**时，UDP 报文段会在数据部分前**临时**加上一个伪首部（Pseudo Header），用于校验数据的完整性。伪首部并不实际发送，只在计算校验和时使用。校验和**可选**，如果**不计算校验和**，置**0**

TCP和UDP的伪首部是一样的，只与IP协议有关 其中伪首部中UDP长度不含伪首部本身

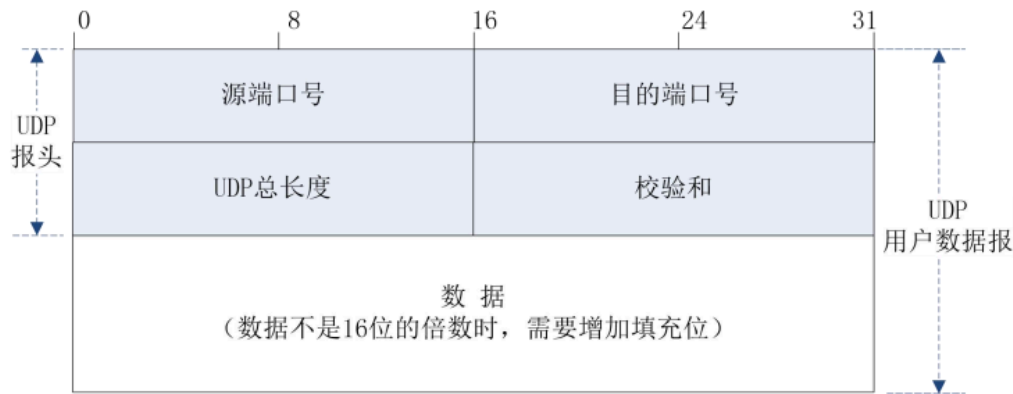
▪ 3.3.2 UDP数据报格式



UDP数据报格式

注意总长度为2字节的倍数

▪ 3.3.2 UDP数据报格式



- 端口号：包括源端口号和目的端口号，分别表示发送方和接收方的进程端口号，各为2字节
- UDP总长度：包括报头在内的用户数据报的总长度，2字节的倍数
- 校验和：用于检查整个数据报（含报头）是否传输出错，可选，若无，填0

UDP校验和计算

将UDP数据报的数据部分以16位为单位划分，相加，若结果大于16位，则进位相加，最后取反，得到校验和。若划分的最后一段不足16位，则用0填充至16位。

可靠数据传输原理

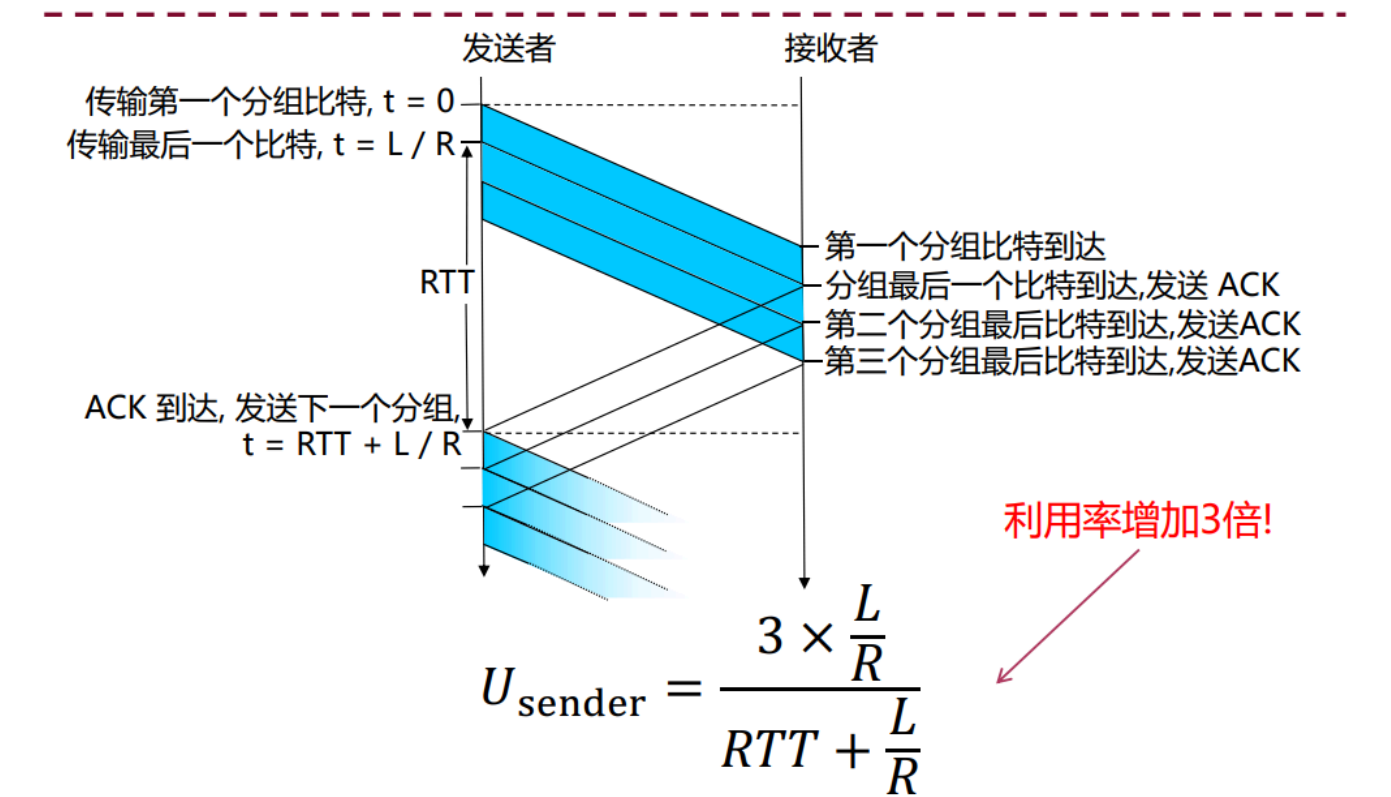
停止等待协议

停等协议：发送方发送一个分组，然后等待接收方的ACK，确认分组到达后，发送方再发送下一个分组。**接受到的ACK序号代表当前已经接受到的序号数**
超时重传：发送方发送一个分组后，启动一个定时器，如果在定时器超时之前没有收到确认分组，则重传该分组。

流水线协议

不用记公式，实际上计算占比只是把数据推上去的时间/除以接受到收到一个数据包的时间
为什么是一个？因为接受数据包实际上是一个瞬间，所以只用算一个

流水线协议: 增加利用率



重传协议

GBN (Go-Back-N, 回退N帧法)

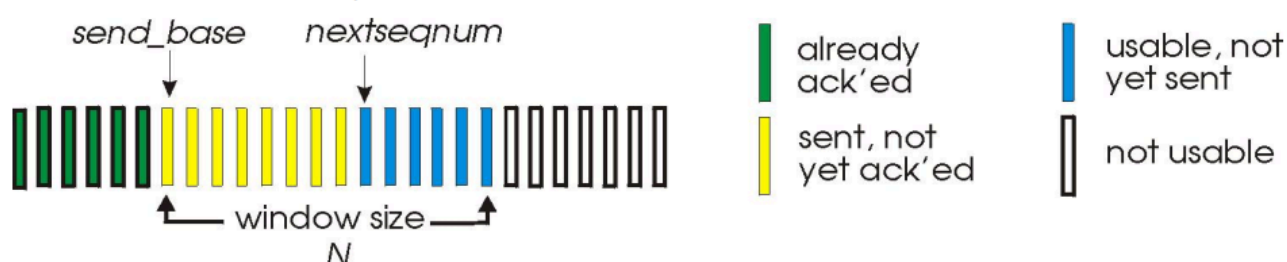
特别注意为了防止串数据, 窗口的大小是 2^k-1

弃置所有不正确分组

Go-Back-N

▪ 发送方:

- 在分组首部需要K比特序号, 序号范围是 2^k ,
- “窗口”最大为N, 允许N个连续的没有应答分组



▪ 发送方必须响应3类事件

- 上层的调用: 检查发送窗口是否已满
- ACK(n): 确认所有的 (包括序号n) 的分组 - “累计ACK”
- timeout(n): 若超时, 重传窗口中的分组n及所有以前的分组。对每个传输中的分组的用同一个计时器

GBN: 接收方扩展 FSM

▪ 对正确接收的分组总是发送具有最高按序序号的ACK

- 可能产生冗余的ACKs
- 仅仅需要记住期望的序号值 (expectedseqnum)

▪ 对失序的分组:

- 丢弃 (不缓存) -> 没有接收缓冲区!
- 用最高的连续序号i重发ACKi报文

SR (Selective Repeat, 选择重传法)

特别注意为了防止串数据，窗口的大小是 $2^{(k-1)}$

只重传丢失的分组,分组间单独计时

选择性重传 (Selective Repeat)

- GBN改善了信道效率，但仍然有**不必要重传**问题
- 选择性重传：
 - 接收方分别确认所有正确接收的报文段
 - 需要缓存报文段, 以便最后按序交付给上层
 - 发送方只需要重传没有收到ACK的报文段
 - 发送方对每个没有确认报文段有一个单独的计时器
- 发送窗口(同GBN)
 - 也需要限制已发送但尚未应答报文段的序号

选择性重传

发送方

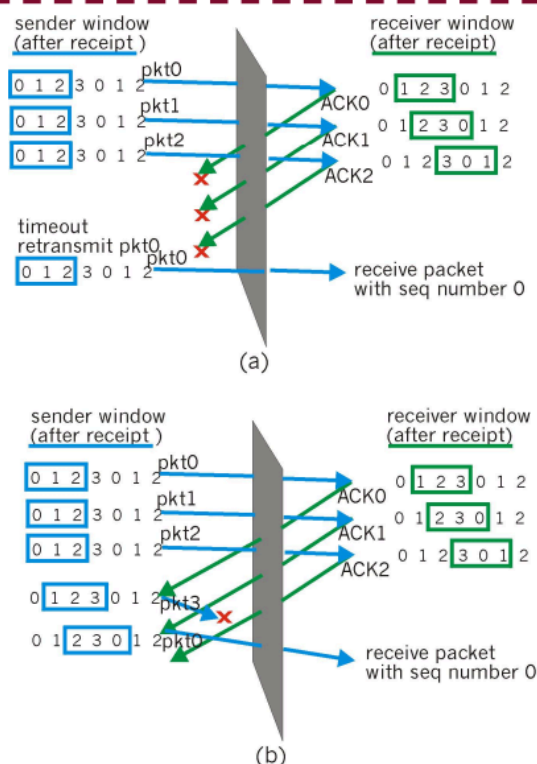
- 上层传来数据：
 - 如果窗口中下一个序号可用, 发送报文段
- timeout(n):
 - 重传分组n, 重启其计时器
- ACK(n) 在[sendbase, sendbase+N]:
 - 标记分组 n 已经收到
 - 如果n 是最小未收到应答的分组, 向前滑动窗口base指针到下一个未确认序号

接收方

- 分组n在 [rcvbase, rcvbase+N-1]
 - 发送 ACK(n)
 - 失序: 缓存
 - 按序: 交付 (也交付所有缓存的按序分组), 向前滑动窗口到下一个未收到报文段的序号
- 分组n在[rcvbase-N, rcvbase-1]
 - ACK(n)
- 其他:
 - 忽略

选择重传: 困难的问题

- 例子: 难以区分新分组还是重传
 - 序号: 0, 1, 2, 3
 - 窗口长度 = 3
- 接收方: 在(a)和(b)两种情况下接收方没有发现差别!
 - 在 (a)中不正确地将冗余的当为新的, 而在(b)中不正确地将新的当作冗余的
- 问题: 序号长度与窗口长度有什么关系?
 - 回答: 窗口长度小于等于序号空间的一半



TCP (报文段)

TCP报文段格式

3.4.2 TCP报文格式



- **发送序号**：若SYN=1,该字段表示TCP数据字段的第1个字节A在当前发送信道T中的初始序号；若SYN=0, 表示A在T中的序号(大于初始序号)
 - 连接建立时，初始序号（ISN）由**随机**数生成器生成，发送端和接收端**独立产生**，可能不一样
 - 占32bit，所能表示的序号范围是：0~(2³²-1)
- **确认序号**：只有当ACK位=1时有效，表示发送此报文段的进程期望接收的下一个新字节的序号。确认序号=N+1，表示接收方已经成功接收了序号为N及之前的所有字节，要求发送方接下来应该发送起始序号为N+1的字节段。
- **窗口大小**：指示发送当前报文段的进程可以接收的数据长度(单位: 字节)。准备接收下一个TCP报文的接收方，通知即将发送报文的发送方下一个报文中最多可以发送的字节数，是发送方确定发送窗口的依据，是动态可变的。

举例：若节点A发送给节点B的TCP报文报头中，确认号字段值为502，窗口字段值为1000，则表示：下一次B向A发送的TCP报文数据字段第一个字节的编号为502，最后一个字节的编号最大为1501
- ****报头长度**：**单位是4字节，表示TCP报文首部的大小；取值范围[5,15] (Why?)
- ****校验和**：**与UDP校验和的相同点：1) 计算方式相同；2) 也需要伪首部。

不同点：1) UDP校验和可选，TCP校验和必须；2) 伪首部协议字段值为6![标志位](image/PixPin_2025-06-28_21-25-55.png)

三次握手、四次挥手

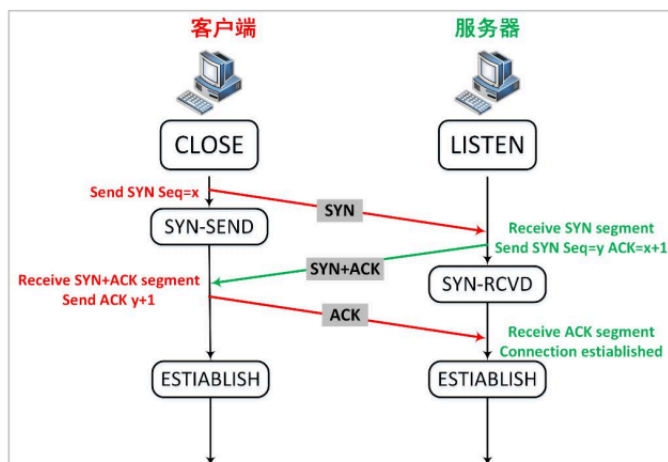
三次握手

注意SYN和FIN都要各自消耗一个序号

3.4.3 TCP基本通信过程

建立连接阶段

- 客户端：发送连接建立请求， $SYN=1$, $ACK=0$, $Seq=x$ 。SYN=1的报文段**不能携带数据**，但要**消耗掉一个序号**
- 服务器端：收到请求并同意， $SYN=1$, $ACK=1$, $Seq=y$, $ackn=x+1$ 。同样该报文段也是SYN=1的报文段，不能携带数据，但同样要**消耗掉一个序号**。
- 客户端：收到服务器的确认，并对此进行确认。 $ACK=1$, $Seq=x+1$, $ackn=y+1$

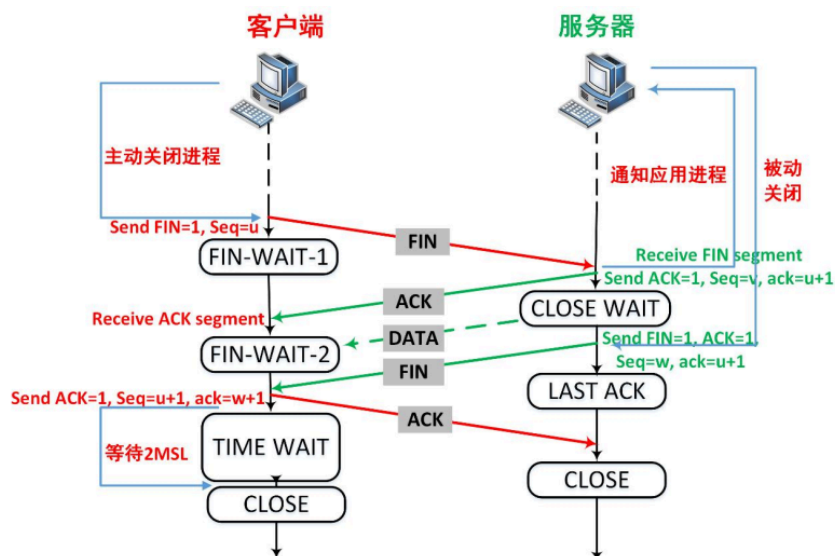


思考：为什么需要三次握手建立连接？两次可不可以？为什么？

四次挥手

释放连接阶段

- 两个方向的连接，可以间隔开来释放
- 每个方向的连接释放，需要2个报文段
- MSL: Maximum Segment Lifetime, 最大报文生存时间，是任何报文段被丢弃前在网络内的最长时间



思考：为什么需要等待2MSL？

尽管有时FIN报文的数据字段长度=0，但FIN报文仍需要消耗额外1个序列号

TCP流量控制

定时器

▪ 3.4.3 TCP基本通信过程

思考以下两个问题：

Q1：假如客户端建立了到服务器的连接，传输了一些数据，但是突发故障崩溃。此种情况下，服务器将处于一种什么状态？

Q2：客户端最终进入的TIME-WAIT状态该如何实现？

保活定时器（服务器端）：每次收到客户端发来的消息就复位；定时器设定为2小时，超时则发送探测包。连续十个探测包还没收到回应，则中断连接。

时间等待定时器（客户端）：设定为报文寿命的2倍

滑动窗口

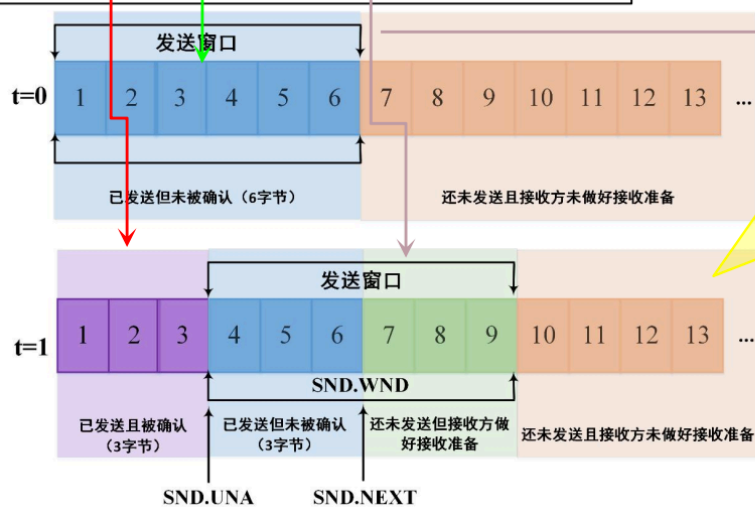
接受窗口可以接受缓存，接受窗口的计算起点在**已经接受到的数据下一个序号**
CP的差错恢复机制可以看成是GBN和SR的混合体：

- 定时器的使用：与GBN类似，只对最早未确认的报文段使用一个定时器

- 超时重传：与SR类似，只重传缺失的数据

3.4.4 滑动窗口与确认重传机制-示例

假定进程A与B建立了一个TCP连接，A向B发送了的报文段覆盖字节编号[1,6]。A收到了B发来的ACK报文段，其中**确认号是4**，**窗口大小是6**字节。



发送方A还可以发送的字节长度
 $= \text{SND.UNA} + \text{SND.WND} - \text{SND.NXT}$
 $= 4 + 6 - 7$
 $= 3$

零窗口

-接收方从缓存中读取速度大于等于字节到达速度，接收方在每个确认中发出一个非零窗口通告

- 如果发送方发送速度比接收方读取速度快，将造成缓冲区被全部占用，之后到达的字节因缓冲区溢出而丢弃。此时，接收方必须发出一个“零窗口”的通告。告知当发送方停止发送（直到接收“非零窗口”通告为止）
- 1. 零窗口(Zero window)通告 -当TCP接收方的缓冲区满(如应用层未能及时提取数据)，则接收方往发送方发送纯ACK报文段(数据长度=0，窗口字段=0)，称之为零窗口通告
- 2. 窗口更新(Window update) -当接收方缓冲区从满状态变到有可用空间时，接收方向发送方发送纯ACK报文段(数据长度=0，窗口字段>0)，称之为窗口更新
- 3. 窗口探测(Window probe) -因为纯ACK报文段(数据长度=0, ACK=1的报文段)不被TCP可靠递送，则第2步的窗口更新有可能在网络中丢失，导致收发双方都处于等待的死锁状态。
 为避免活锁，发送方使用一种叫**坚持定时器(persist timer)**来定期触发发送方往接收方发送窗口探测报文段(数据长度>0, 保证被TCP递送)。作为响应，接收方将自己的缓冲区可用空间大小放入ACK报文段的窗口字段，由此，发送方获知接收方是否能继续接收数据。

RTT估计

3.4.4 TCP滑动窗口与超时重传机制

$$\text{Timeout} = \beta \times \text{RTT}$$

其中

- Timeout: 本次定时器的时间值
- RTT: 对本次发出报文段的往返时间的估计值
- β : 常数值, 大于1, 推荐设置为2

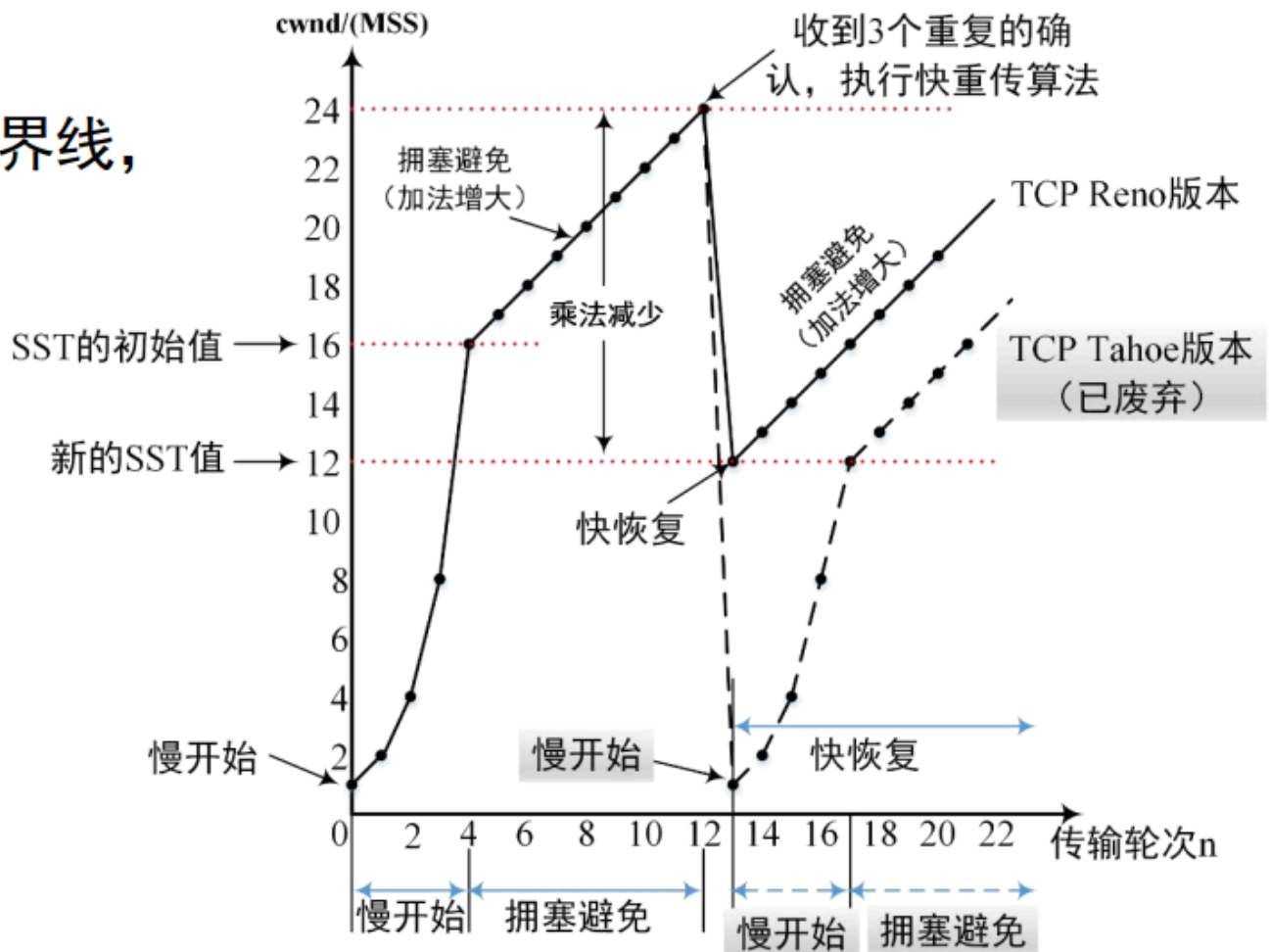
$$\text{RTT} = \alpha \times \text{旧RTT} + (1 - \alpha) \times \text{最新测量RTT}$$

其中

- α : 常数值, 小于1, 推荐设置为0.125

TCP拥塞控制

分界线,



- MSS: 最大报文段长度，TCP报文段中数据字段的**最大长度**,是**拥塞控制的单位**，默认568字节，但是一般题目设定
- 慢开始：在该阶段，cwnd初始值为1，且每结束一个RTT，cwnd翻倍当 **cwnd=SSThresh**时进入拥塞避免阶段
- 拥塞避免：在该阶段，cwnd每过一个RTT增加1
- 拥塞判定：
当收到**3个重复的ACK**时，判定发生拥塞，SSThresh置为cwnd/2，然后cwnd一起置为一半，即**快恢复**，直接开始拥塞避免阶段 当超时时，判定发生拥塞，SSThresh置为cwnd/2，然后cwnd一起置为一，开始慢开始阶段
- 快重传：收到3个重复ACK时，立即重传丢失的报文段，是一个操作，本身**不计入时间**

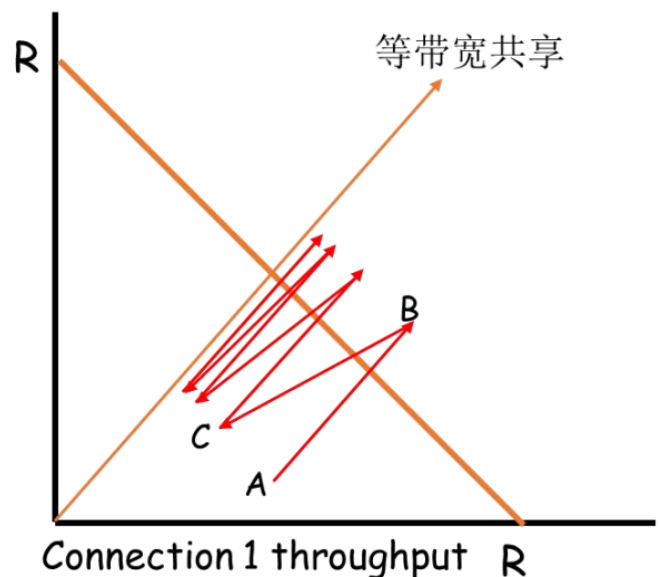
发送窗口 = Min (通知窗口(rwnd), 拥塞窗口(cwnd))

公平性

即在公平性原则下，TCP连接的带宽应当被公平地分配给所有的连接。

为什么TCP是公平的

- 考虑两条竞争的连接（各种参数相同）
共享带宽为R的链路：
- 加性增：连接1和连接2按照相同的速率增大各自的拥塞窗口，得到斜率为1的直线
- 乘性减：连接1和连接2将各自的拥塞窗口减半



网络层（分组(Packet)）

功能

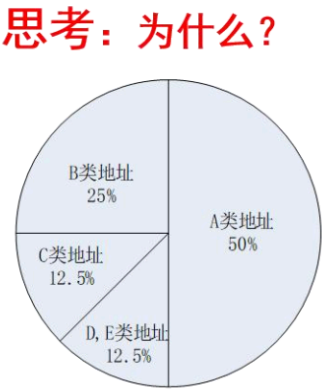
IP地址

分类

- **A类地址 (0)**: 1.0.0.0~126.255.255.255, 1~126, 127为回环地址
- **B类地址 (10)**: 128.0.0.0~191.255.255.255, 128~191
- **C类地址 (110)**: 192.0.0.0~223.255.255.255, 192~223
- **D类地址 (1110)**: 224.0.0.0~239.255.255.255, 224~239, 用于多播
- **E类地址 (11110)**: 240.0.0.0~255.255.255.255, 240~255, 保留
 - 标准分类: 网络号+主机号, 根据网络号前缀和位数的差异, 分为A、B、C、D和E五类



a) 标准分类的IP地址



b) 每类地址比例数量

特殊IP地址

- **0.0.0.0|全0**: 本地主机
- **255.255.255.255|全1**: 广播地址
- **127.0.0.1**: 本地回环地址

网络号全0或全1的地址, 以及网络号为10的地址留作专用地址, 还剩下125个地址块; 主机号全0或者全1的地址用于特殊目的, 故每个A类子网可以分配的主机号为

特殊IP地址

网络号	主机号	源地址使用	目的地址使用	含义
0	0	可以	不可	在本网络上的本主机（可用于 DHCP 协议）
0	host-id	可以	不可	在本网络上的某台主机 host-id
全 1	全 1	不可	可以	只在本网络上进行广播（各路由器均不转发）
net-id	全 1	不可	可以	对 net-id 上的所有主机进行广播
127	非全 0 或全 1 的任何数	可以	可以	用作本地软件环回测试之用

专用IP地址

- 10.0.0.0~10.255.255.255： 10.0.0.0/8
- 172.16.0.0~172.31.255.255： 172.16.0.0/12
- 192.168.0.0~192.168.255.255： 192.168.0.0/16

子网划分

记得网络号不能全0（子网号不能全0），全1（子网号不能全1）

子网掩码（Subnet Mask）

- A类地址： 255.0.0.0
- B类地址： 255.255.0.0
- C类地址： 255.255.255.0 从高到低位连续的1，直到遇到第一个0为止 子网掩码和 IP地址按位与运算，得到网络号 有10： 128， 110:192， 1110:224， 1111:240

CIDR

无类域间路由选择（CIDR） 是一种IP地址分配方法，旨在提高IP地址的使用效率。CIDR允许将多个IP地址块聚合为一个单一的路由条目，从而减少路由表的大小。CIDR表示法使用斜杠后跟一个数字来表示网络前缀的长度，例如192.168.0.0/24表示网络前缀

为24位的子网掩码255.255.255.0。
效果上和子网掩码差不多

NAT

在网络层插入 NAT（NetworkAddressTranslation）后，IP 报文在进出私有网络时会被“篡改”一下，具体来说，NAT 会对以下字段做修改，并相应地重计算校验和：

1. IP 层的修改

被改字段	说明
源／目的 IP 地址	私有网段（如 192.168.x.x）翻译成公网 IP，或反向翻译成内网 IP。
IP 头校验和	IP 首部发生变化后，原有的 IP 首部校验和（HeaderChecksum）失效，必须重算。

2. 传输层的修改

- 端口号（仅在 PAT／NAPT 情况下）
 - 当多台私有主机共用一个公网 IP 时，NAT 会把每个私有主机的源端口（TCP/UDP 段头的 SourcePort）映射到一个唯一的公网端口，用于区分不同会话。
- TCP/UDP 校验和
 - 因为 IP 地址或端口变了，伪首部（pseudo header）也跟着变，必须重新计算 One’ s-complement 校验和，才能让接收端验算通过。

3. 应用层的潜在影响

某些协议会把 IP 地址硬编码在应用数据中（比如 FTP、SIP、某些点对点协议），NAT 直接改报文头并不会触及它们的“内嵌地址”，这时就需要 **ALG（Application-Level**

Gateway) 或 STUN/TURN 等机制额外帮忙翻译或穿透。

4. NAT 带来的副作用

- 1. **破坏端到端连接**：中间设备改了地址/端口，端到端安全或认证（IP SEC、TLS 客户端绑定 IP）可能失效。
- 2. **增加延迟和状态维护开销**：NAT 设备要为每条流维护转换表，并定期清理超时条目。
- 3. **多对一映射冲突**：需要精心设计端口分配策略，避免不同内网会话映射到同一公网端口。

简单流程示意



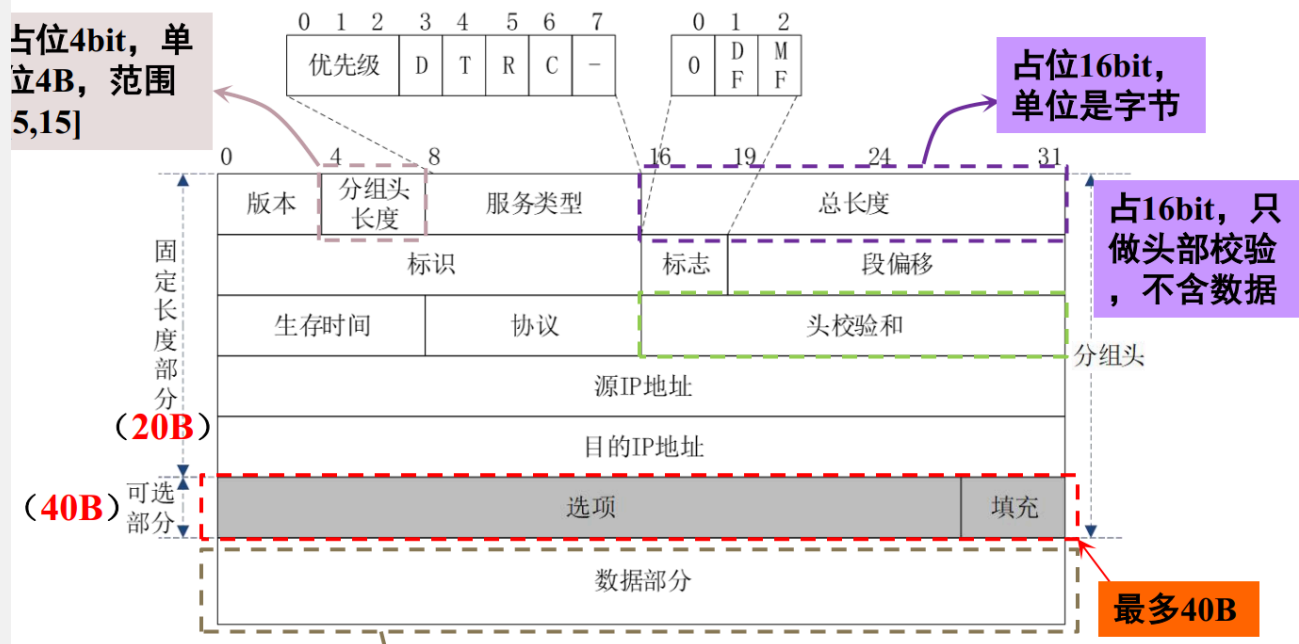
总结：

NAT 会改变报文的 IP 地址和（在端口地址转换时）端口号，并为此重算 IP 校验和和 TCP/UDP 校验和；对嵌入式应用地址要额外配合 ALG 或穿透技术。

IP格式

IP长度为20字节到60字节，固定长度20字节

IPv4分组结构



第一行

版本字段:

占4bit, 4代表IPv4, 6代表IPv6

分组头长度: 占4bit,

单位4B, 范围[5,15]

总长度:

占16bit, 单位是字节, 表示分组头长度和数据长度之和

第二行

标识:

占16bit, 用来标识 不同的IP分段, 最多能 分配2¹⁶-1个ID

标志:

占3bit, 最高位 固定为0, 中间位为DF, 最低位为MF

- DF=1，表示接收节点不能对分组进行分段。反之，则可
- MF=0，表示接收的是最后一个分段。反之，则不是

段偏移：

占13bit，表示分段在整个分组中的相对位置，以8字节为单位进行计数，因此分段长度应为8字节的整数倍

第三行

生存时间TTL：

占8bit，表示分组在网络中的存活时间，以跳数来计数

- 避免分组在网络中循环转发
- 当TTL=0时，丢弃分组

协议：

占8bit，表示IP的高层协议类型，可选值如表中所示

头部校验和：

用于头部校验

- 不负责数据校验
- 降低延迟，提高效率

MTU

含义：链路帧的数据字段的最大长度，也就是封装链路帧时允许的**IP分组大小上限**

特点：

- （1）不同网络的MTU大小不同；
- （2）规定IP分组的最大长度为65535字节；
- （3）MTU一般小于IP分组长度，因此需要将IP分组分割成若干个较小的段，每个段的长度不超过MTU

分组分段

IP分组长度超过MTU时，需要分段

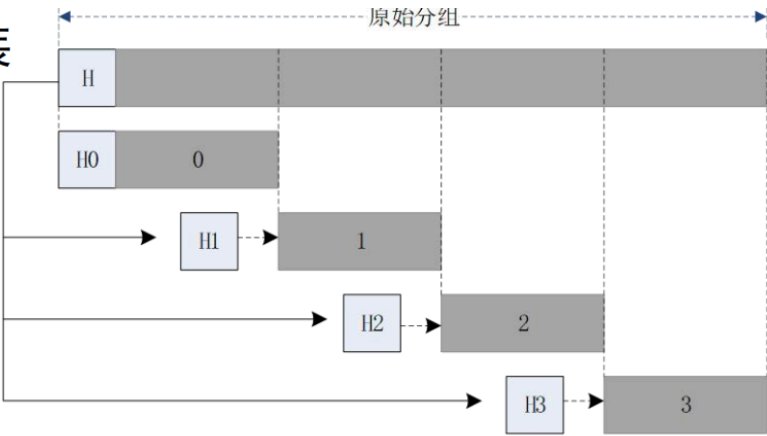
- 分段后，每个分段的IP头部会复制一份，并重新计算校验和
- 分段后的IP分组在到达目的地后，需要重新组装

务必注意里面的MF和DF

例子1

4.4.2 IPv4分组的分段与组装

例：一IP分组的数据长度为2200字节，头部长度的为20字节，MTU大小为820字节。试问是否需要分段？如需，怎么分段？

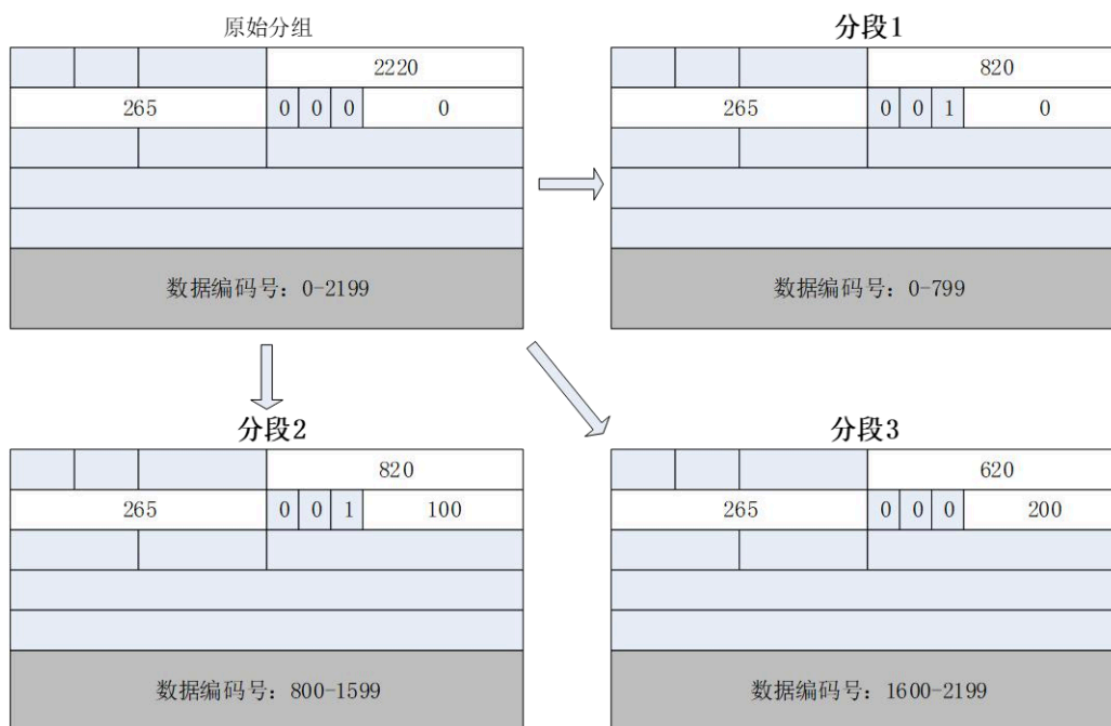


原始分组:	分组头	段1 (800B)	段2 (800B)	段3 (600B)
段1:	分组头	段1 (800B)	段偏移=0; DF=0; MF=1	
段2:	分组头	段2 (800B)	段偏移=100; DF=0; MF=1	
段3:	分组头	段3 (600B)	段偏移=200; DF=0; MF=0	

思考：段偏移、DF、MF值是如何计算的？

例子2

4.4.2 IPv4分组的分段与组装



例子3

4.4.2 IPv4分组的分段与组装-随堂练习题

1. 一个IP分组的长度为3000B（固定首部长度）。现在经过一个网络传送，但此网络能够传送的最大数据长度为1020B。试问应当划分为几个分组段？每段的总长度、数据字段长度、段偏移字段和MF标志应为什么数值？

答：分为3个数据报片

原始数据报：总长度3000B，数据长度2980B MF=0，段偏移为0

第一个数据报片：总长度 1020B，数据长度1000B，MF1，段偏移为0

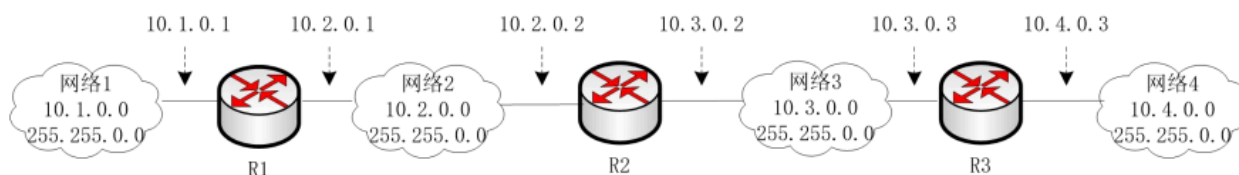
第二个数据报片：总长度 1020B，数据长度1000B，MF1，段偏移为125

第三个数据报片：总长度 1000B，数据长度980B，MF0，段偏移为250

IP路由选择

路由形式

■ 子网路由选择算法



R2的路由表

子网掩码	目的网络	下一跳路由器
255.255.0.0	10.2.0.0	直接转发
255.255.0.0	10.3.0.0	直接转发
255.255.0.0	10.1.0.0	10.2.0.1
255.255.0.0	10.4.0.0	10.3.0.3

子网路由选择算法:

➤ 路由表三元组 (M, N, R), M是目标网络子网掩码

➤ 取出目的地址, 逐项与路由表中的子网掩码做位与操作, 得到目的子网地址, 找到匹配条目

静态路由、动态路由

静态路由选择算法:

将到每个目的地址的路径都固定存储在路由表中, 唯一地确定了从源主机到目的主机的路径

- 路由表无法自动更新; 路由表的更新由管理员手动完成

动态路由选择算法:

网络系统运行时, 系统自行运行动态路由选择协议 建立路由表

- 网络结构变化时, 可自动更新路径

路由聚合|最长前缀匹配

简单来说, 就是通过CIDR, 将同前缀、同接口的IP地址合并

为什么能够聚合? 因为有了CIDR之后, 如果IP地址的子网掩码后相同会认为匹配同一个路由, 所以可以合并

4.5 路由选择算法与分组转发

4.5.1 基本概念-IP路由汇聚

路由器	输出接口
156.26.63.240/30	S0（直接连接）
156.26.63.244/30	S1（直接连接）
156.26.63.0/28	S0
156.26.63.16/28	S1
156.26.0.0/24	S0
156.26.1.0/24	S0
156.26.2.0/24	S0
156.26.3.0/24	S0
156.26.56.0/24	S1
156.26.57.0/24	S1
156.26.58.0/24	S1
156.26.59.0/24	S1

0 = 00000000
1 = 00000001
2 = 00000010
3 = 00000011

4项的最长相同的前缀是22位。因此，路由表中的这4项可以合并成：156.26.0.0/22

56 = 00111000
57 = 00111001
58 = 00111010
59 = 00111011

4项的最长相同的前缀是22位。因此，路由表中的这4项可以合并成：156.26.56.0/22

路由器	输出接口
156.26.63.240/30	S0（直接连接）
156.26.63.244/30	S1（直接连接）
156.26.63.0/28	S0
156.26.63.16/28	S1
156.26.0.0/22	S0
156.26.56.0/22	S1

路由协议

RIP(Bellman-Ford算法) (UDP)

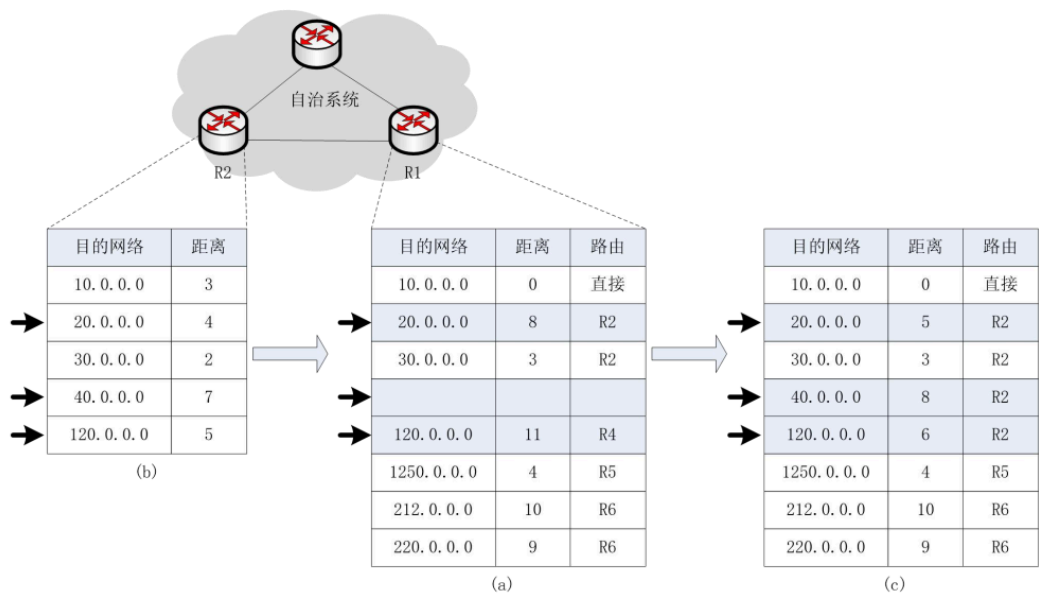
用UDP的原因是RIP仅仅向相邻路由器发送路由信息，相对可靠性高，故用UDP

特别注意路由向量中距离大于等于16的表明路由不可达！！！！

- 设置周期更新定时器，每隔30s在相邻路由器之间交换一次路由信息
- 如果接收到同一网络有多条距离相同的路径，将按照“先入为主”原则，取用第一条路径信息（直到该路径失效，或被更短路径取代）
- 根据向量距离路由选择算法，当有开销小的路径出现时，修改表中路由记录，否则一直保持下去
- 每个路由表项设有超时定时器，在路由表项被修改时开始计时，若180s后没有收到刷新信息时，表示该路径已经出现故障，将该项纪录置为“无效”（不删除该

项路由记录)

▪ 4.5.3 路由信息协议RIP



路由表信息的更新过程

OSPF (Dijkstra算法)

用IP的原因是OSPF需要向所有路由器发送路由信息，IP方便广播
在链路状态发生变化时，用洪泛法向所有路由器发送该信息
路由器之间交换链路状态信息有：费用、距离、延时、带宽等信息

- 每个路由器都要维护一张链路状态数据库，该数据库中存储了整个自治系统的拓扑结构
- 每个路由器根据链路状态数据库，计算出到达每个目的地的最短路径，并将结果写入路由表中

内部算法对比

算法类型	协议	特点	通信方式	传递内容
距离向量算法	RIP使用	每个节点只和邻居节点进行通信	邻居间通信	传递到所有节点的距离（以该点为目的的最短路径距离）
链路状态算法	OSPF使用	每个节点和其余各个节点都进行通信	全网通信	只传递与其直接相连的链路状态

BGP（外部网关）

自治系统（AS）

- 互联网由自治系统互联而成，“自治”管理（大学、公司、部门等），有权自主地定内部采用何种路由协议
- 域内路由选择：自治系统内部的路由选择
- 域间路由选择：不同自治系统之间的路由选择 BGP的基本思想
- 不同AS路由器之间交换路由信息
- 采用路径向量路由协议（不同于RIP、OSPF协议）
- **每个AS至少有一个路由器作为该AS的“BGP发言人”**
- 考虑安全、经济方面因素，寻找**较好的路由**，而不是最佳路由

收敛性好

ICMP（IP直装）

功能：检查、纠正、查询、反馈
有请求报文和应答报文

检查

Ping 检查是否联通 超时、错误、不可达

查询

时间戳、路由、地址掩码

源点抑制

在 IP 层之上，ICMP（Internet Control Message Protocol）提供了一组“差错报告”和“诊断”消息，其中之一就是 **源点抑制（Source Quench）**，用于在网络拥塞时通知发送端“放慢发送速率”。

3. 工作流程

1. 触发

- 如果路由器或目标主机的输出接口缓冲区积压过多、队列接近满载，就可能对入站数据触发 Source Quench。

2. 发送

- 拥塞节点向报文的源 IP 发出 ICMP Type4、Code0 报文。

3. 响应

- 收到 Source Quench 的源主机应当：
 - 降低发送速率，例如增加发送窗口（TCP 中减小拥塞窗口 cwnd）、加大重传定时、延长两次报文的间隔。
 - 持续监听，直到不再收到抑制消息后，再逐步恢复原有发送速率。

种类

报文类型	种类
差错报告	目的站不可达
	数据报超时
	数据报参数错
	源站抑制
	路由重定向
查询报文	时间戳请求与应答
	回应请求与应答
	地址掩码请求与应答
	路由器查询与通告

IPV6

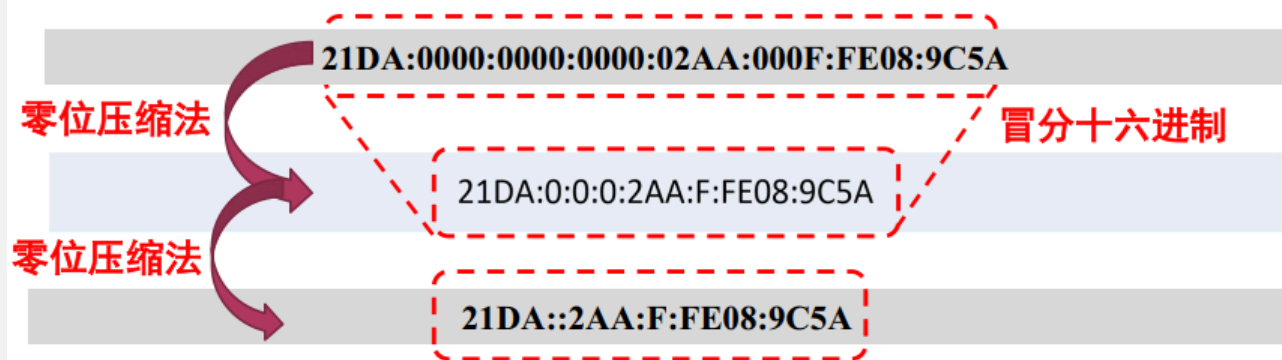
结构

基础头、扩展头、数据

压缩

注意只能有一个::，首部不能完全空着，空掉的0都在数字前部

➤ 冒分十六进制地址表示法



- 在使用零压缩法时，**不能**将一个位段的**有效0**压缩掉。例如，不能将 `FF02:30:0:0:0:0:0:5` 简写为 `FF02:3::5`，而应该简写为 `FF02:30::5`
- 冒号在一个地址中只能出现一次

CIDR

IPV6的CIDR和IPV4的CIDR是一样的

特殊地址

下面列出一些在 IPv6 中常见的、拥有特殊用途的地址类型，并简要说明它们的作用和地址范围：

地址类型	地址前缀	示例	作用说明
未指定地址	::/128	::	表示“无地址”或“尚未配置地址”，只能用作源地址，不能作为目标地址。
环回地址	::1/128	::1	本机环回——相当于 IPv4 的 127.0.0.1，用于本地进程间通信。
链路本地地址	fe80::/10	fe80::a00:27ff:fe4e:66a1	每个接口自动生成，用于同一链路（子网）内的通信，不可路由到 Internet。
IPv4 映射地址	::ffff:0:0/96	::ffff:192.0.2.128	用于在 IPv6 应用中表示 IPv4 地址，便于双栈过渡。
多播地址	ff00::/8	ff02::1、ff02::2 等	用于一对多通信，不同范围和作用由后面的标志和范围字段决定。

简要使用场景

- :: / ::1：系统初始化或本地测试时用。
- ::ffff:....：过渡期内双栈应用，用来表示传统 IPv4 主机。

IPv6与IPv4的相同点

字段名	IPv4 报头	IPv6 报头	语义／作用
Version	4 位	4 位	协议版本号，IPv4 为 4，IPv6 为 6；路由器和主机根据此字段识别报文格式。
服务类型／流量控制	Type of Service (ToS)	Traffic Class 8 位	用于区分不同服务等级（DiffServ）和显式拥塞通知（ECN）。

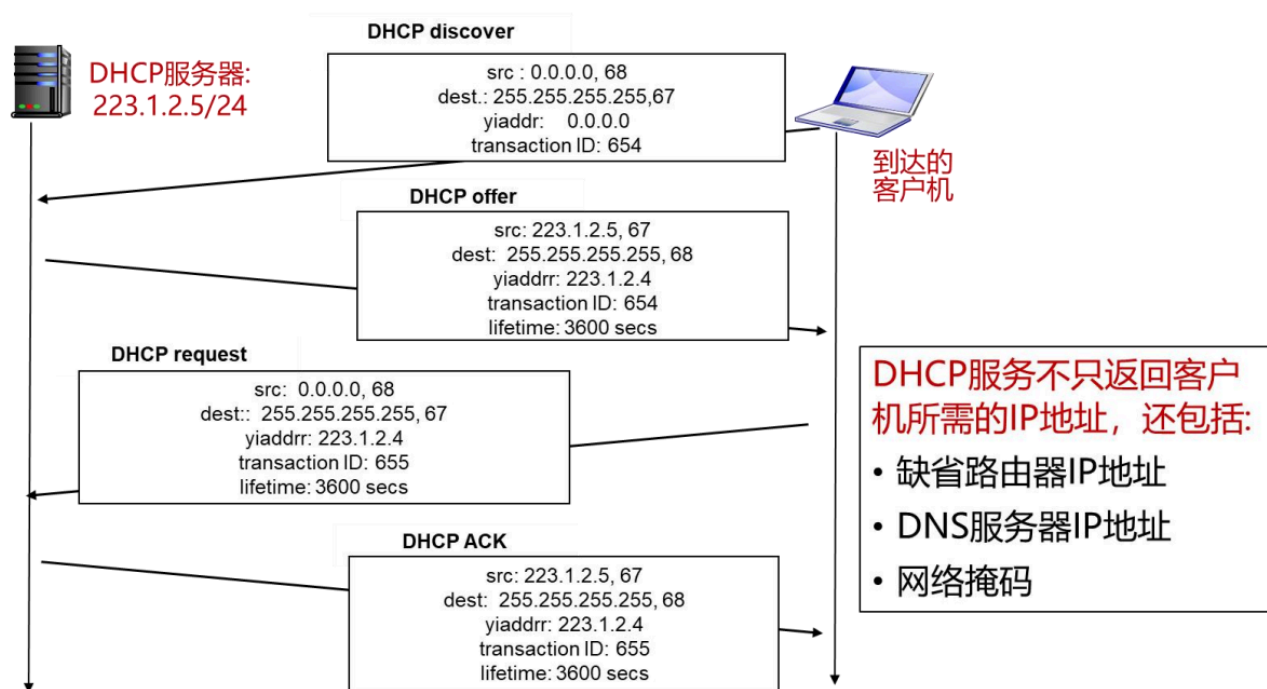
字段名	IPv4 报头	IPv6 报头	语义／作用
	8 位 (DSCP + ECN)	(DSCP + ECN)	
报文长度	Total Length 16 位	Payload Length 16 位	IPv4 中包含头部+数据总长度；IPv6 中只包含后续“负载”部分长度（固 定 40B 头之外）。
协议／下一 个头	Protocol 8 位	Next Header 8 位	指明载荷使用的上层协议（TCP=6、 UDP=17 等）或下一个扩展头类型。
生存时间／ 跳数限制	Time To Live (TTL) 8 位	Hop Limit 8 位	每转发一次递减 1，用完即丢弃；防 止分组在网络中永久循环。
源地址	32 位	128 位	发起端的 IP 地址，用于返回路径选择 和上层协议鉴别。
目的地址	32 位	128 位	分组要到达的目标地址，用于路由查 表并最终交付给目的主机。

DHCP

▪ DHCP工作过程

- DHCP 客户从UDP端口68以**广播形式**向服务器发送发现报文（**DHCPDISCOVER**）
- DHCP 服务器**广播/单播**发出提供报文（**DHCPOFFER**）
- DHCP 客户从多个DHCP服务器中选择一个，并向其以**广播形式**发送DHCP请求报文（**DHCPREQUEST**）
- 被选择的DHCP服务器**广播/单播**发送确认报文（**DHCPACK**）

▪ DHCP工作过程



数据链路层（帧）

帧

封装成帧(framing)就是在一段数据的前后分别添加首部和尾部，然后就构成了一个帧。

确定帧的界限

- 首部和尾部的一个重要作用就是进行帧定界

重传(同网络层的GBN和SR,也接受ACK)

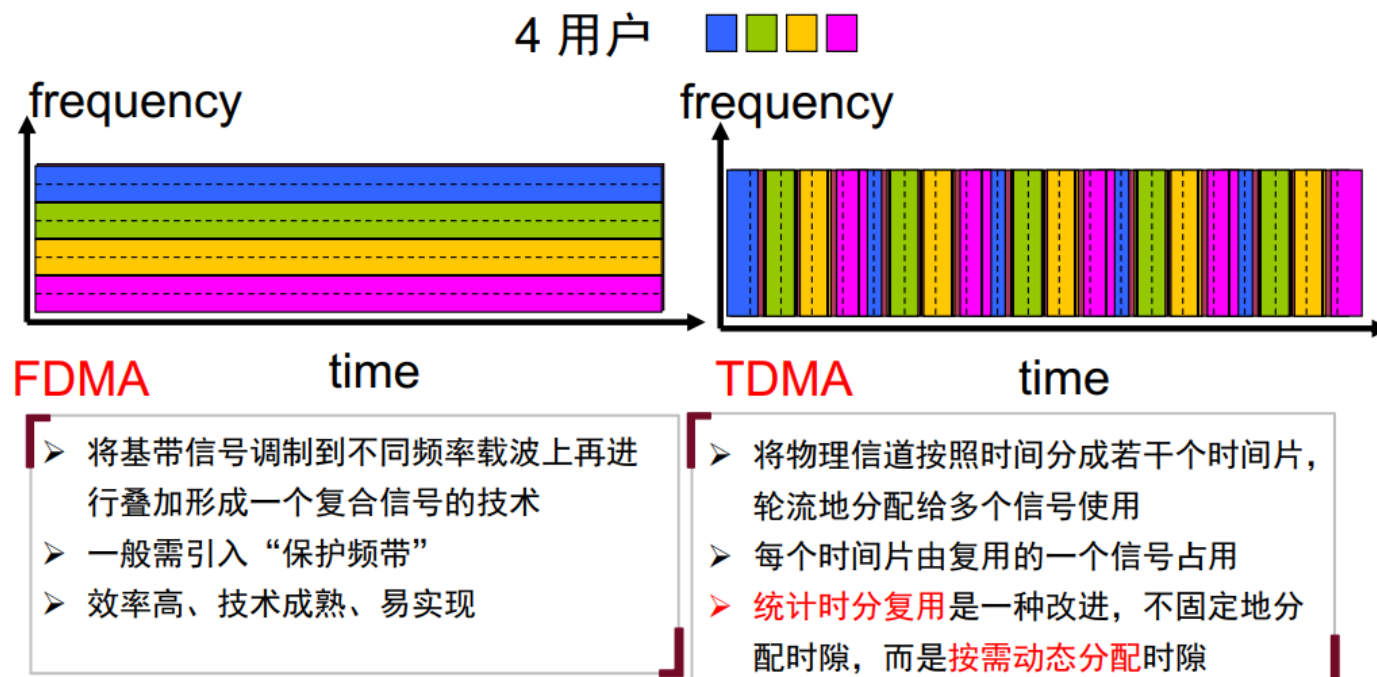
一、链路层的复用（多址／多路访问）

链路层要让多路信号共享同一物理媒介，常用三种基本复用方式：

方式	英文	原理	优缺点
时分复用	TDM (Time-Division Multiplexing)	把时间划成固定时隙，不同信号占用不同时隙传输	+ 简单 - 时隙空闲浪费 - 固定延迟
频分复用	FDM (Frequency-Division Multiplexing)	把频谱划成多个子信道，不同信号占不同频段并行传输	+ 并行无时延 - 频率资源固定难动态调整 - 需滤波器
码分复用	CDM/CDMA (Code-Division)	所有信号并行用同一频带，用相互正交的【扩频】码区分	+ 灵活动态 + 抗干扰 - 需同步与正交码管理

5.5 局域网参考模型与以太网工作原理

5.5.2 信道划分介质控制访问—频分与时分多路复用



复用量

在链路层／物理层，用不同方式把多路信号复用到同一条介质上时，常常需要计算“复用量”——也就是在给定资源（时隙、频带、码片率）下，最多能承载多少路信号。简单来说就是总的部分/划分的一小部分

1. 时分复用（TDM）

- 资源：一个周期帧的总时长 T_{frame}
- 每路时隙：时隙时长 T_{slot}
- 复用量（时隙数）：

$$M_{\text{TDM}} = \left\lfloor \frac{T_{\text{frame}}}{T_{\text{slot}}} \right\rfloor$$

- 每路带宽：如果信道总速率是 R (bps)，则每路平均可用带宽

$$r = \frac{R \times T_{\text{slot}}}{T_{\text{frame}}}$$

示例：E1 系统帧长 $125\mu\text{s}$ ，30 个语音时隙+2 个时隙控制，共 32 时隙，

$$M = \frac{125\mu\text{s}}{125\mu\text{s}/32} = 32.$$

2. 频分复用 (FDM)

- **资源：**信道总带宽 B_{total}
- **每路带宽：**通道带宽 B_{ch} （包括保护带）
- **复用量（子信道数）：**

$$M_{\text{FDM}} = \lfloor \frac{B_{\text{total}}}{B_{\text{ch}}} \rfloor$$

示例：若有 1GHz 光纤带宽，每个无线频道占 5MHz（含保护带），

$$M = \lfloor 1000\text{ MHz}/5\text{ MHz} \rfloor = 200.$$

3. 码分复用 (CDM/CDMA)

- **资源：**扩频带宽 W 与用户速率 R ；处理增益 $G_p = W/R$
- **系统容纳用户数（理想情况下）：**

$$M_{\text{CDMA}} \approx 1 + \frac{G_p}{(\frac{E_b}{N_0})_{\text{req}}}$$

- 其中 $(E_b/N_0)_{\text{req}}$ 是用户速率下维持误码率所需的比特能量噪声功率比。

示例：若 $W = 5\text{ MHz}$ ，用户 $R = 10\text{ kbps}$ ，则 $G_p = 500$ 。如果每用户需 $(E_b/N_0)_{\text{req}} = 10\text{ dB}$ (≈ 10)，

$$M \approx 1 + \frac{500}{10} = 51 \text{ 用户}.$$

为什么公式长这样？

- **TDM**：把时间分成固定时隙，能有多少就能承载多少。
 - **FDM**：把频谱平分给各路，带宽够划就能划。
 - **CDMA**：所有用户共用全带宽，用正交（伪随机）码区分，容量受“处理增益”与“噪声要求”约束。
-

单工、半双工与全双工

- **单工**：只能一个方向通信，如广播。
- **半双工**：能双向通信，但不能同时双向，如对讲机。
- **全双工**：能同时双向通信，如电话。

二、链路层的“邻接传输”

- **作用**：链路层仅在**直接相连的两个节点**之间传送帧，屏蔽底层物理差异，提供：
 1. **帧定界**（Frame Delimiting）
 2. **地址寻址**（MAC 地址）
 3. **差错检测**（CRC 校验）
 4. **重发控制**（若支持 ARQ）
 - **典型技术**：以太网、令牌环、Wi-Fi、PPP 等都只在“相邻链路”生效。
-

三、点对点协议：PPP

PPP (Point-to-Point Protocol) 是最常用的点对点链路层协议

1. 帧结构

- Flag (0x7E) | Address (0xFF) | Control (0x03) | Protocol (16b) | Payload | FCS (16/32b)

2. 点到点信道：信道**直接连接两个端点**（PPP）

3. 多点访问信道：多用户共享一根信道（**以太网**）

4. 特点：

- 能够在不同的链路上运行
- 能够承载不同的网络层分组
- 特点： 简单、灵活

四、链路层差错控制：ARQ

在链路层也可做可靠传输，常见 ARQ（自动重传请求）策略：

策略	原理概述
停止等待	发送一帧后等 ACK，再发下一帧，丢包重传；效率低（窗口=1）。
Go-Back-N	发送窗口 $W>1$ ，若超时或三次重复 ACK，重传从失序处到当前所有帧。
Selective Repeat	异步确认并缓存乱序帧，只重传丢失帧；效率高但实现复杂。

以太网(以下涉及到的概念都是以太网相关的)

以太网在链路层的位置 链路层 包含两部分：

- 逻辑链路控制（LLC）：帧的定界、错误检测、多协议多路复用（802.2）
- 介质访问控制（MAC）：具体介质的访问与帧格式（802.3）

以太网规范即是 IEEE 802.3 MAC + PHY，在链路层提供：

- 地址寻址：MAC → 上层 IP 选路
- 错误检测：CRC 校验，检出位翻转或噪声错误
- 流量和冲突管理|媒体访问控制方式：CSMA/CD 或交换全双工（交换机）

MAC

局部网络地址,不越过路由器|网关

五、典型共享介质多址协议

下面几种协议都用于“多节点共享同一链路”场景：

协议	原理 / 特点
纯ALOHA	任意时刻随发帧，若冲突则发送端等待随机时间后重发；
时隙ALOHA	将时间划成一段段的时隙,只能在时隙的起点发帧，冲突仍需重发；
CSMA/CD	载波监听+冲突检测（以太网）,冲突重试；适有线。
CSMA/CA	载波监听+碰撞避免（Wi-Fi）；先听信道、若空闲则延迟随机DIFS/SIFS后发；适无线。

- **ALOHA** 简单易实现，但效率低，不适合高密度网络。
- **CSMA/CD** 适合双绞线或同轴，多节点线性拓扑；但不适用于无线（无法可靠检测碰撞）。
- **CSMA/CA** 通过“先听后发”+随机退避+确认应答等机制，降低无线环境下冲突概率。

简单说,ALOHA是任意时刻发,CSMA是先听后发,然后CD有线,CA无线

ARP|RARP

ARP(IP-->MAC)

RARP(MAC-->IP)

纠错

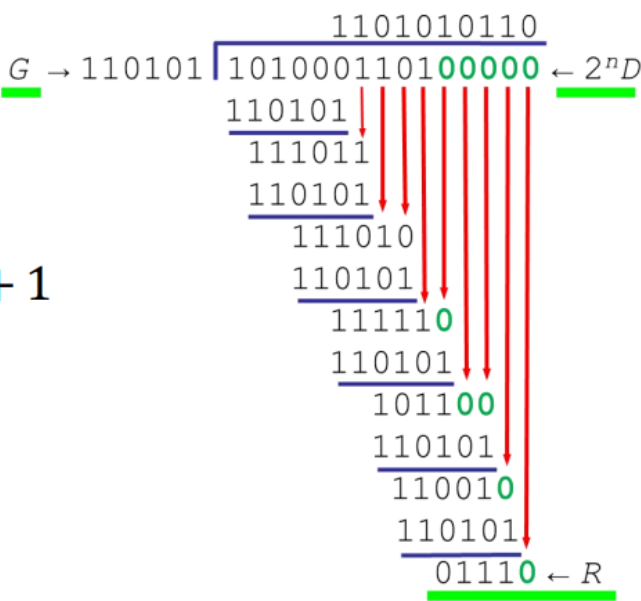
CRC

必考

5.2.3 差错的检测与纠正-循环冗余校验码

CRC校验码计算示例

- $D = 1010001101$
- $n = 5$
- $G = 110101$ 或 $G = x^5 + x^4 + x^2 + 1$
- $R = 01110$
- 实际传输数据: 101000110101110



奇偶校验

5.2.3 差错的检测与纠正—奇偶校验码

- 海明码：可以检错并纠正以为差错的编码，信息位足够长时，编码效率高
- 奇偶校验：在信息位后面加上一位校验位，使得码字（信息位+校验位）中“1”的个数为奇（偶）数 **【发送端编码】**
- 监督式： $S = a_{n-1} \oplus a_{n-2} \oplus a_{n-3} \cdots \oplus a_1 \oplus a_0$ 。偶校验，当 $S = 0$ 时，无错； $S = 1$ 时，出错。奇校验：反之。 S 为校正因子 **【接收端校验】**

sender				receiver			
数据位			校验位	数据位			校验位
0	0	0	0	0	0	0	0
0	0	1	1	1	0	1	0
0	1	0	1				
0	1	1	0				
1	0	0	1				
1	0	1	0				
1	1	0	0				
1	1	1	1				

error

1	1	0	0
0	1	0	1

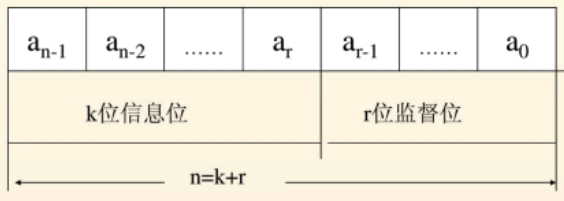
奇偶校验法只能检测奇数个错误

海明码

5.2.3 差错的检测与纠正—海明码

- 利用一个以上的校验位，不仅可以验证数据是否有效，还能在数据出错的情况下指明错误位置
- 若用 r 个监督关系式产生 r 个校正因子，区分无错、码字中 n 个不同位置的位错，则要求 $2^r \geq n + 1$ ，也即 $2^r \geq k + r + 1$

举例： $k=4$ ，信息位1101，要满足上述要求，根据 $2^r \geq n + 1$ 可得到 $r \geq 3$ 。取 $r = 3$ ，则 $n = 7$ 。信息位 $a_6a_5a_4a_3$ ，冗余位 $a_2a_1a_0$ 。 a_2 、 a_1 和 a_0 由某几位信息位半加所得



- 码字 $n \text{ (bit)} = k + r$
- k : 信息位
- r : 冗余位

1. 误码率（Bit Error Rate, BER）

- 定义：在数字通信中，误码率是指接收端错误比特数与总传输比特数之比：

$$\text{BER} = \frac{\text{误码比特数}}{\text{总传输比特数}}$$

- 意义：衡量链路或系统的“可靠性”——BER 越低，说明传输越可靠。
- 影响因素：
 - 通道噪声、干扰强度；
 - 调制方式（BPSK、QPSK、QAM 等对噪声的敏感度不同）；
 - 误码检测／纠错机制（如前向纠错 FEC）。

2. 信噪比（Signal-to-Noise Ratio, SNR）

- 定义：信号功率与噪声功率之比，通常以线性值或对数 dB 表示：

$$\text{SNR} = \frac{P_{\text{signal}}}{P_{\text{noise}}} \quad \text{或} \quad \text{SNR}_{\text{dB}} = 10 \log_{10}(P_{\text{signal}}/P_{\text{noise}}) \text{ dB}$$

- **意义：**衡量信号的“清晰度”——SNR 越高，系统检测或解调信号的难度越小，误码率通常也越低。
- **香农极限：**在带宽为 B (Hz)、信噪比为 SNR 的理想信道中，最大无差错传输速率为

$$C = B \log_2(1 + \text{SNR}) \quad (\text{bps})$$

- **BER 与 SNR 的关系：**不同调制下大致关系如下（以 BPSK 为例）：

$$\text{BER}_{\text{BPSK}} = Q(\sqrt{2E_b/N_0}) = \frac{1}{2} \text{erfc}(\sqrt{E_b/N_0})$$

其中 E_b/N_0 与 SNR 成正比。

二、二进制指数退避算法 (Binary Exponential Backoff)

主要用于 CSMA/CD 和 Ethernet 的冲突重传中，用于决定每次冲突后等待的退避时隙数：

1. 初始设置

- 定义“时隙时间” (slot time)，例如以太网 10Mbps 环境下为 $51.2 \mu\text{s}$ 。

2. 冲突计数 k

- 每次检测到冲突，冲突计数 k 加 1，直到达到最大允许值（通常 10 或 16）。

3. 随机退避

- 计算候选退避时隙数 N 为：

$$N \in [0, 2^k - 1] \quad (\text{均匀随机选取整数})$$

4. 等待重发

- 等待 N 倍的“时隙时间”后再次尝试发送。

5. 上限与放弃

- 当 k 超过预设最大冲突次数（例如 16）时，放弃本次帧并上报错误。

算法流程图示意

```
冲突计数  $k = 0$ 
重传尝试:
  如果检测到冲突:
     $k = \min(k+1, k_{\max})$ 
    随机  $N \in [0, 2^k-1]$ 
    等待  $N \times \text{slot\_time}$ 
    回到“重传尝试”
  否则:
    成功发送,  $k = 0$  (重置)
```

物理层(比特流)

调制|解调

调制 (Modulation)

将数字信号转换为一种特定的模拟信号，以便在传输过程中能够有效地传输信息。调制的过程包括改变信号的特征参数，如振幅、频率或相位，从而将原始信号嵌入到一个载波信号中。

- 数字信号→模拟信号

解调 (Demodulation)

是指将调制后的信号还原为原始信号的过程。在接收端，解调器会识别和提取出被调制的信号，并将其转换回原始信号的形式，以便接收方能够正确解读和处理信息。

- 模拟信号→数字信号

调制方法

调幅 (Amplitude Modulation, AM)

通过改变载波的振幅来传输数字信号。在调幅中，数字信号的“1”和“0”通过改变载波的振幅来表示。

调频 (Frequency Modulation, FM)

通过改变载波的频率来传输数字信号。在调频中，数字信号的“1”和“0”通过改变载波的频率来表示。

调相 (Phase Modulation, PM)

通过改变载波的相位来传输数字信号。在调相中，数字信号的“1”和“0”通过改变载波的相位来表示。

脉冲编码调制 (Pulse Code Modulation, PCM), 重要!!!

将模拟信号转换为数字信号的过程。PCM 通过对模拟信号进行**采样**、**量化**和**编码**来实现。

步骤是采样、量化、编码!!

数字信号编码

1. 非归零编码 (Non-Return-to-Zero, NRZ)

- NRZ-L: 1为高电平, 0为低电
- NRZ-I: 1为电平翻转, 0为电平不变

2. 归零编码 (Return-to-Zero, RZ)

- RZ: 1为高电平, 0为低电平, 且电平在中间时归零

3. 曼彻斯特编码 (Manchester Encoding)

- **Manchester**: 1为电平翻转, 0为电平不变

4. 差分曼彻斯特编码 (Differential Manchester Encoding)

- **DManchester**: 1为电平翻转, 0为电平不变, 但电平翻转在中间时归零

传输介质

1. 双绞线 (Twisted Pair Cable)

- **双绞线**: 由两根绝缘导线组成, 它们被绞合在一起以减少电磁干扰。
- **屏蔽双绞线**: 在双绞线外加上一层金属屏蔽层, 以减少外部电磁干扰。

2. 同轴电缆 (Coaxial Cable)

- **同轴电缆**: 由一根中心导线、一层绝缘层、一层金属屏蔽层和一层绝缘层组成。
- **细同轴电缆**: 中心导线较细, 适用于短距离传输。
- **粗同轴电缆**: 中心导线较粗, 适用于长距离传输。

3. 光纤 (Fiber Optic Cable)

- **光纤**: 由玻璃纤维或塑料纤维制成, 用于传输光信号。
- **单模光纤**: 只能传输一种模式的光信号, 适用于长距离传输。
- **多模光纤**: 可以传输多种模式的光信号, 适用于短距离传输。

4. 无线传输介质

- **无线电波**: 通过无线电波进行传输。
- **微波**: 通过微波进行传输。

香农定理

香农定理（Shannon 定理、香农-哈特利定理）是信息论的基石，由克劳德·香农于 1948 年提出，主要阐明了有噪声信道的最大无差错传输速率。

一、信道容量公式

对于带宽受限且加性高斯白噪声信道，香农-哈特利公式给出了信道的容量（最大可靠传输速率）：

$$C = B \log_2(1 + \text{SNR}) \quad (\text{bps})$$

- C ：信道容量，单位比特每秒 (bps)
- B ：信道带宽，单位赫兹 (Hz)
- $\text{SNR} = \frac{P_{\text{signal}}}{P_{\text{noise}}}$ ：信噪比（线性值）

四、简要示例

假设带宽 $B = 1 \text{ MHz}$ ，信噪比 $\text{SNR} = 10$ ($\approx 10 \text{ dB}$)，则

$$C = 10^6 \times \log_2(1 + 10) \approx 10^6 \times 3.46 = 3.46 \times 10^6 \text{ bps}$$

即在该信道上，理论上最可靠的数据速率约为 3.46 Mbps。

复用

1. 频分复用 (Frequency Division Multiplexing, FDM)

- FDM**：将带宽分成多个子信道，每个子信道传输不同的信号。

2. 时分复用 (Time Division Multiplexing, TDM)

- TDM**：将时间分成多个时隙，每个时隙传输不同的信号。

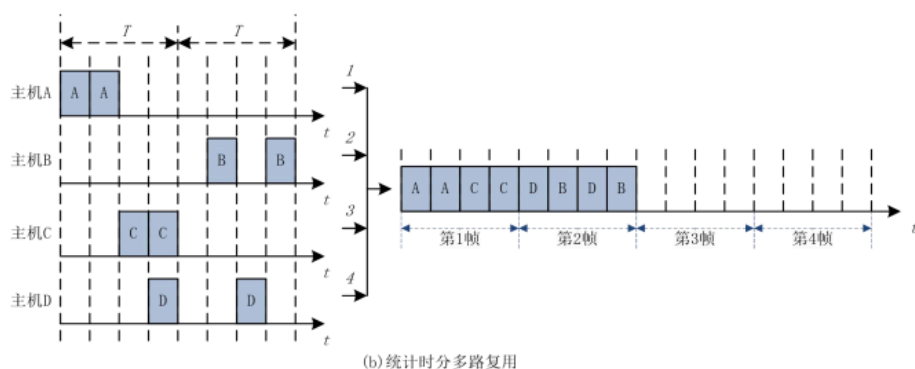
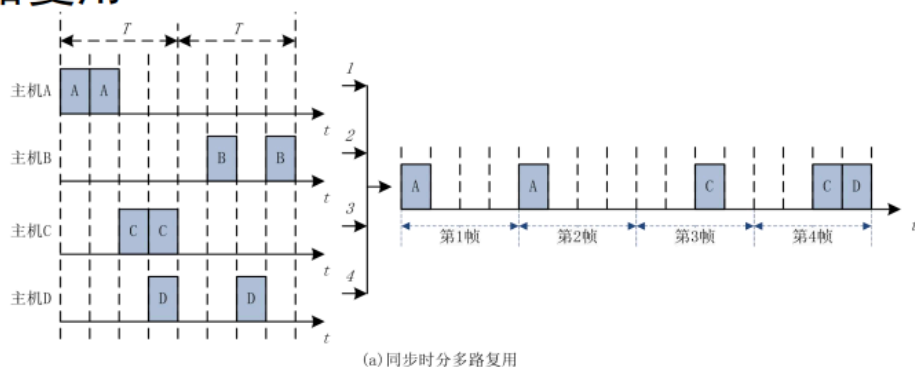
同步时分多路复用

- 将时间片预先分配给各个信道，时间片固定不变，各个信道发送和接收 必须同步，即在同一帧内四个信道同时发送

统计时分多路复用

- 统计时分多路复用：**异步时分多路复用， 允许动态地分配时间片,即统计发送后根据实际需求分配发送帧

■ 时分多路复用



3. 波分复用 (Wavelength Division Multiplexing, WDM)

- WDM：**将光信号分成多个波长，每个波长传输不同的信号。